Talk for SYCL MODE

KHRONOS GROUP
CONNECTING SOFTWARE TO SILICON

SYCL Programming and Performance Optimization

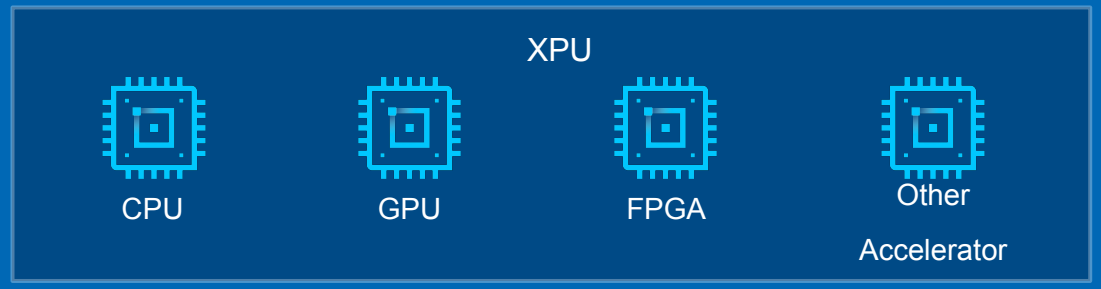
Patric Zhao, Intel

intel

Outline

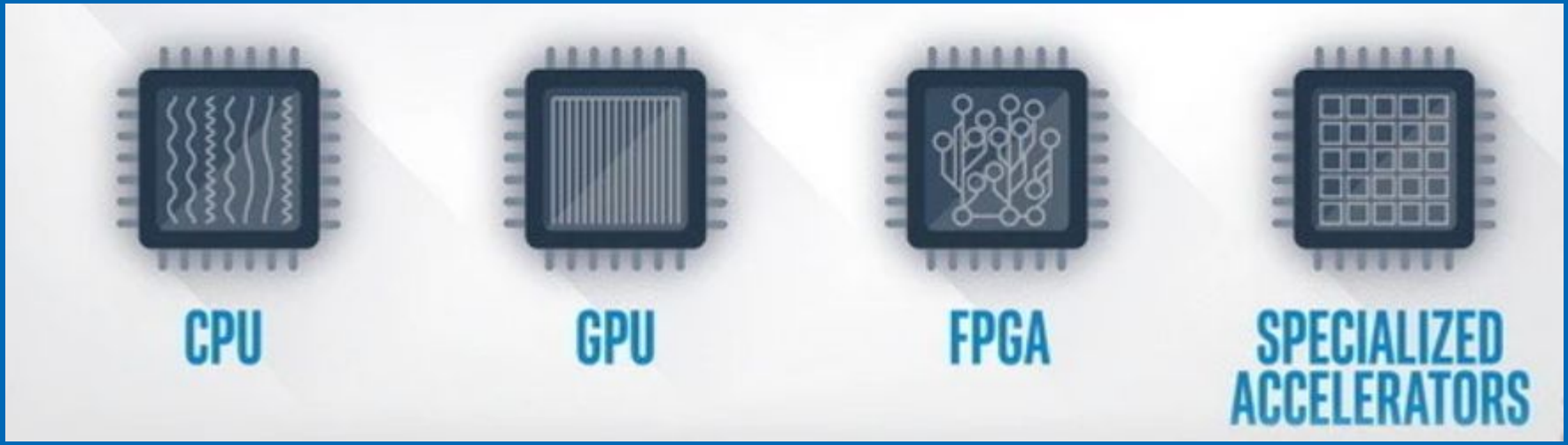
- Quick Review of Heterogeneous Computing
- SYCL Programming
  - *Basic Parallel Kernels*
  - *Execution And Memory Model*
- Performance Optimization with Case Study

Heterogeneous Computing



Introduction

Heterogenous Computing: A computing system and approach that is comprised of different computing units which is based on different instruction set and system architecture.



General Purpose Processor: CPU + GPU for acceleration

CPU

- General Purpose Tasks
- Most Common Applications
- Compute Intensive

GPU

- Specialized tasks（GEMM, Conv）
- Graphics and Vision Applications
- Computation in parallel
- High memory bandwidth

Execution model: independence

- Physical separated, inter-connect via mainboard
- Independent memory space
- Different computing logic and instructions

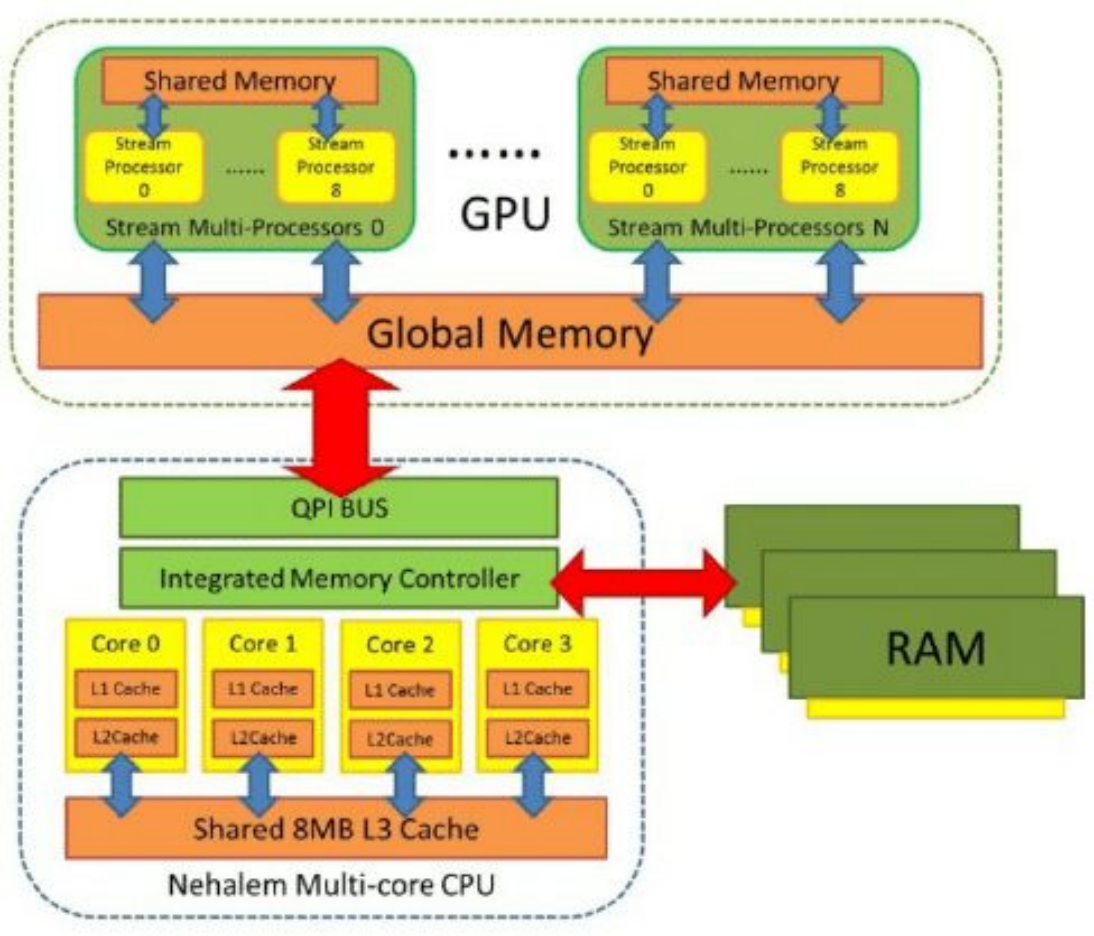
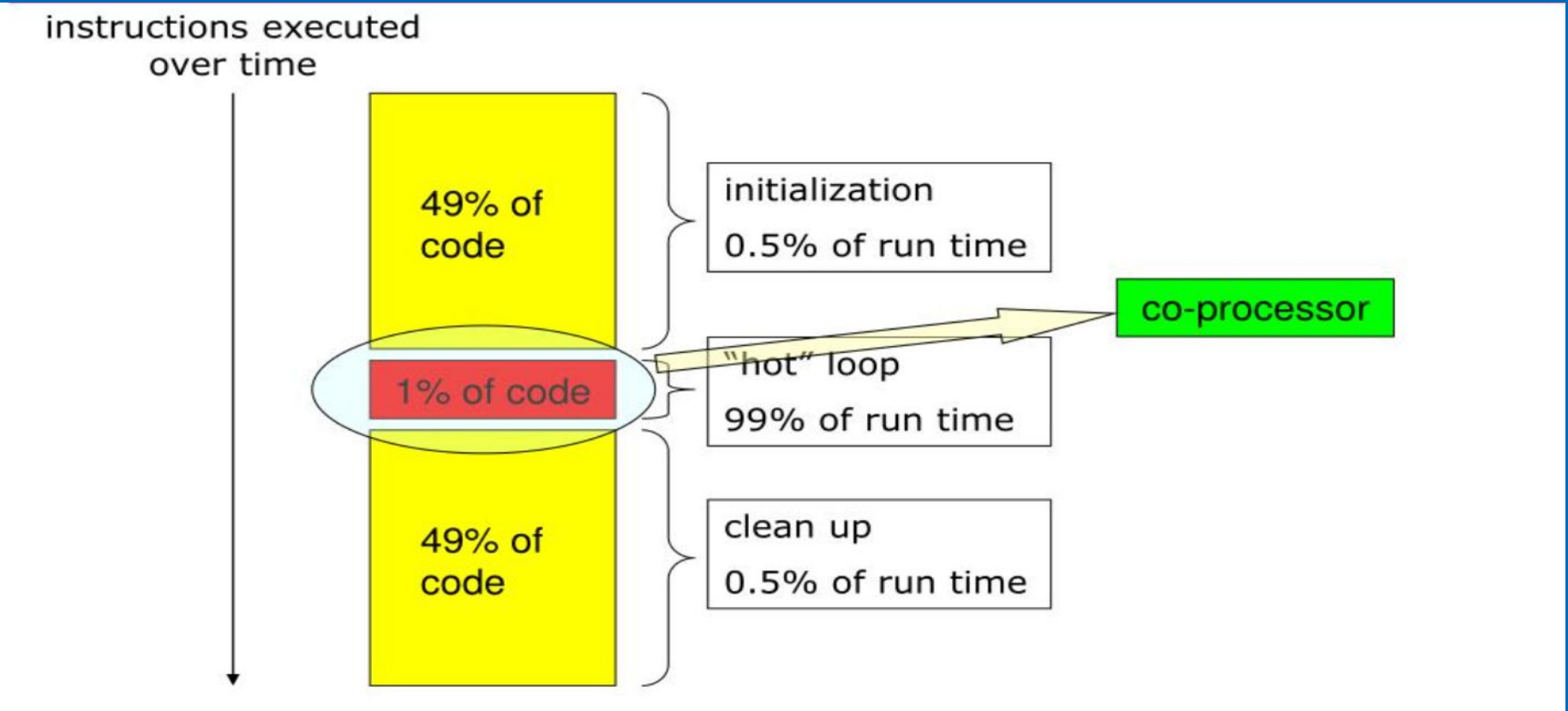


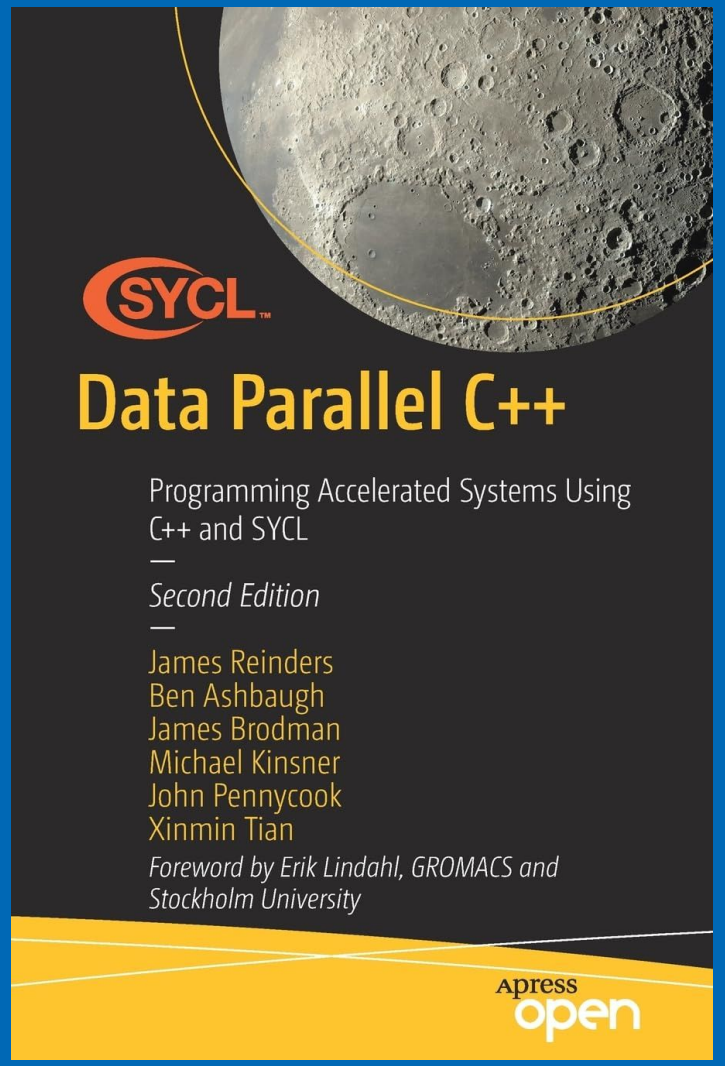
Image Source: https://bmcsystbiol.biomedcentral.com/articles/10.1186/1752-0509-6-S1-S16/figures/3

Execution model: dependence



KHRONOS GROUP
CONNECTING SOFTWARE TO SILICON

SYCL Programming Language



https://github.com/Apress/data-parallel-CPP

What and Why SYCL?

It is an extension to C++ language, providing:

- SYCL (pronounced ‘sickle’) is a free, cross-platform abstraction layer.
- Enables code for heterogeneous and offload processors to be written using modern ISO C++ (at least C++ 17).
- Provides APIs and abstractions to find devices (e.g. CPUs, GPUs, FPGAs) on which code can be executed, and to manage data resources and code execution on those devices.

Codeplay oneAPI Plug-ins for NVIDIA and AMD

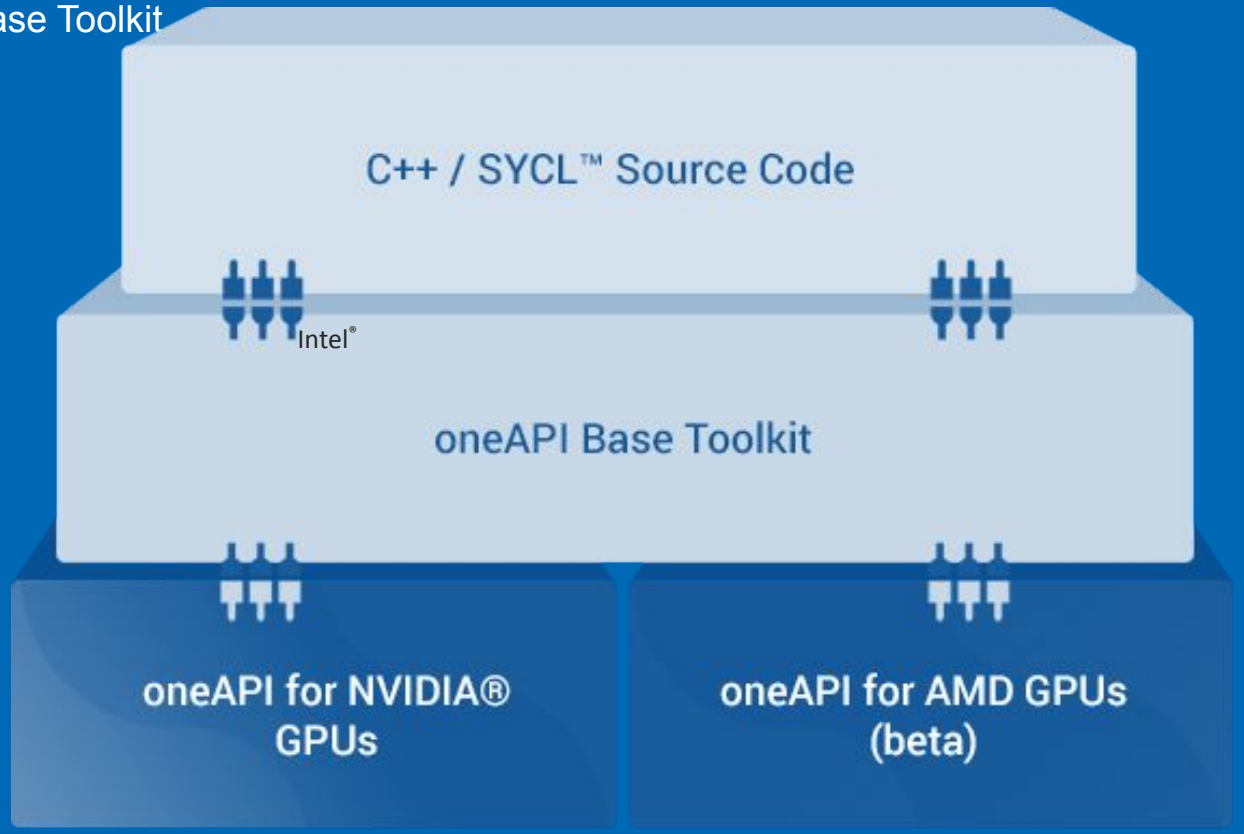
Support for NVIDIA and AMD GPUs to Intel® oneAPI Base Toolkit

oneAPI for NVIDIA and AMD GPUs

- Free download of binary plugins to Intel® oneAPI DPC++/C++ Compiler:
  - NVIDIA GPU
  - AMD beta GPU
  - No need to build from source!
  - Plug-ins updated quarterly in-sync with SYCL 2020 conformance and performance

Priority Support

- Available through Intel, Codeplay, and our channel
- Requires Intel Priority Support for Intel oneAPI DPC++/C++ Compiler
- Intel takes first call, Codeplay delivers backend support
- Codeplay provides access to older plug-in versions



NVIDIA GPU plug-in | AMD GPU plug-in

Codeplay blog | Codeplay press release

Other names and brands may be claimed as the property of others.
SYCL is a trademark of the Khronos Group Inc.

Image courtesy of Codeplay Software Ltd.

How SYCL works?

It is an extension to C++ language, providing:

- Abstractions for accessing hardware devices
- Methods for data access
- Methods for expressing parallelism

Devices

- **Devices,** represent various hardware in the oneAPI system

  The device class is a predefined method for selecting and querying devices

  It contains member functions to query device information

  It supports the creation of different hardware, CPU/GPU/FPGA/…

- The **device_selector** class supports selecting a specific device at runtime

  default_selector, cpu_selector, gpu_selector...

Code Sample

```
#include <sycl/sycl.hpp>
#include <iostream>
using namespace sycl;
int main() {
  queue my_gpu_queue( gpu_selector{} );
}
```

```
patric@gpu:~$ icpx -fsycl gpu_selector.cpp
patric@gpu:~$ ./a.out
Selected GPU device: Intel(R) Iris(R) Xe MAX Graphics [0x4905]
```

https://github.com/pengzhao-intel/oneAPI_course/blob/main/code/gpu_selector.cpp

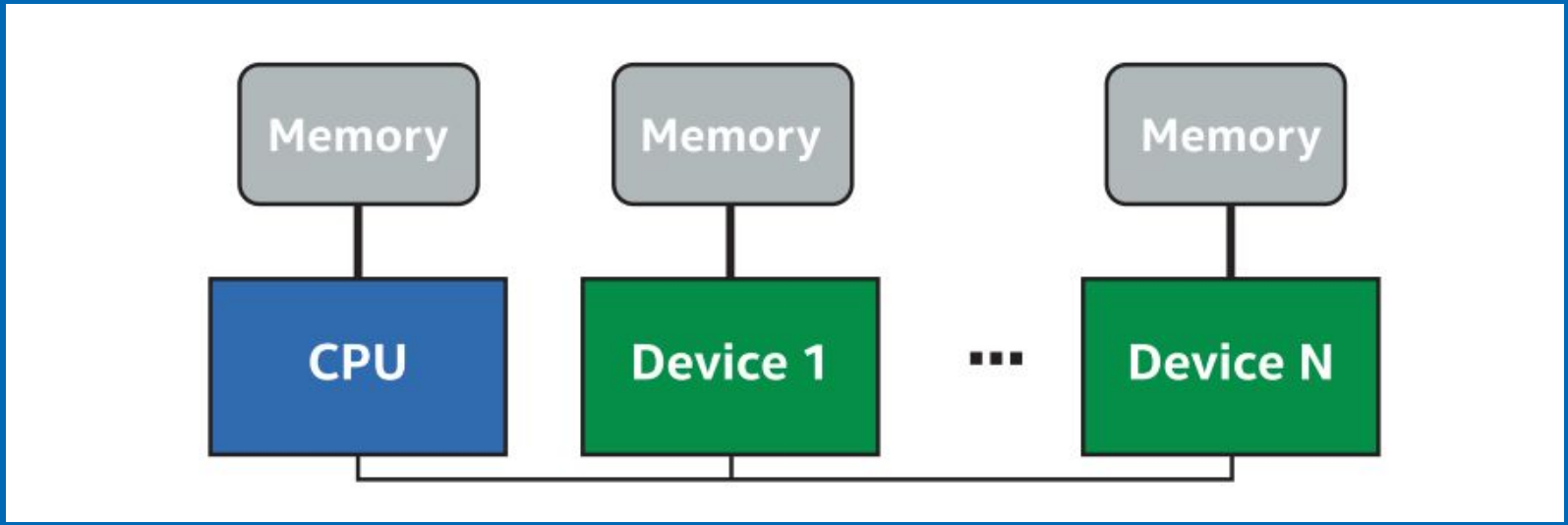
Queue

- Queue is a mechanism for submitting work to the device
- The main program pushes the task into the queue, and the heterogeneous device obtains the execution task from the queue
- A queue is map to a device, and multiple queues can map to the same device

```
int main() {
  queue my_gpu_queue( gpu_selector{} );
  std::cout << "Selected GPU device: " <<
    my_gpu_queue.get_device().get_info<info::device::name>() << "\n";
  return 0;
}
```

Data Movement

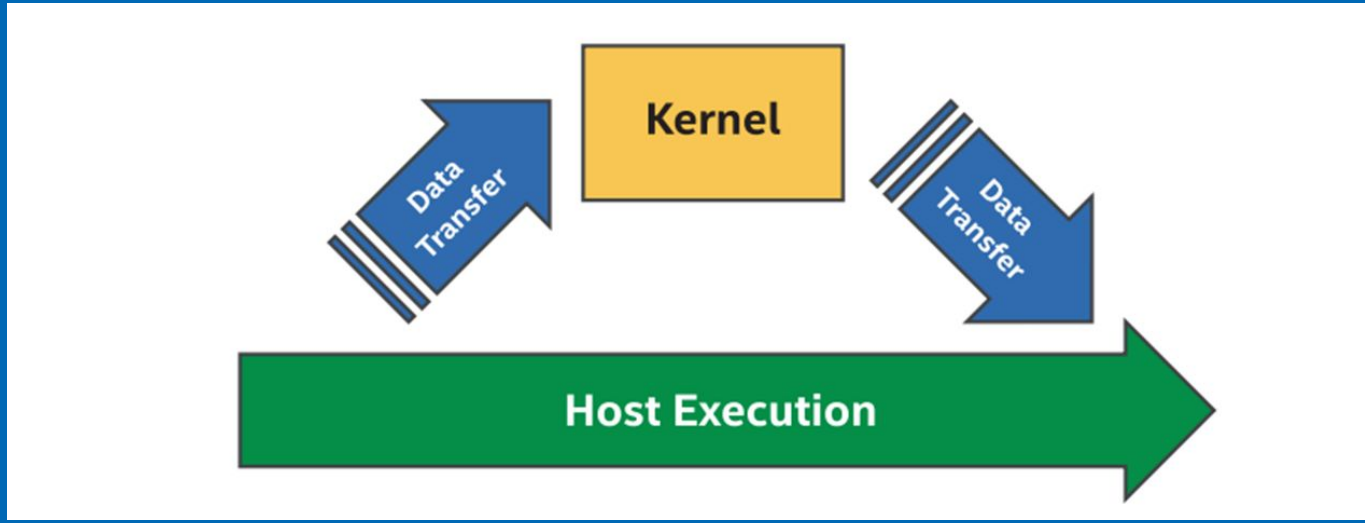
- CPU and GPU have independent memory space



- Programmers need to consider how to transfer data between different devices

- Explicit data transfer: CPU ->GPU, computing, GPU->CPU

- Implicit data transfer: data is automatically migrated by the system when accessed



• Explicit memory allocation

Traditional C/C++

void* malloc (size_t size);

Allocate memory block

Allocates a block of *size* bytes of memory, returning a pointer to the beginning of the block.

Data Parallel C++

void* malloc_host(size_t size, const sycl::queue& q);

void* malloc_device(size_t size, const sycl::queue& q);

void* malloc_shared(size_t size, const sycl::queue& q);

• Explicit memory copy

Traditional C/C++

void * memcpy (void * destination, const void * source, size_t num);

Copy block of memory

Copies the values of *num* bytes from the location pointed to by *source* directly to the memory block pointed to by *destination*.

Data Parallel C++

void * queue.memcpy (void * destination, const void * source, size_t num);

Code Sample

```
constexpr int N = 10;

  int *host_mem   = malloc_host<int>(N, my_gpu_queue);
  int *device_mem = malloc_device<int>(N, my_gpu_queue);
```

Compilation:

patric@gpu:~/course$ ipcx -fsycl data_movement_ex.cpp -o data_move

Run & output:

patric@gpu:~/course$ ./data_move

Selected GPU device: Intel(R) Iris(R) Xe MAX Graphics [0x4905]

Data Result

0, 1, 2, 3, 4, 5, 6, 7, 8, 9,

Task Done!

wait();

wait();

https://github.com/pengzhao-intel/oneAPI_course/blob/main/code/data_movement_ex.cpp

Kernel

- Which parts needs to be parallelized?

The most computationally intensive and time-consuming part

usually within loops

Serial execution

```
for(int i=0; i < 1024; i++){
    a[i] = b[i] + c[i];
});
```

Parallel execution

```
launch N kernel
instances {
    int id =
get_instance_id();
    c[id] = a[id] +
b[id];
```

parallel_for

- The parallel for-loop is the core in parallel computing.

  In this loop, each iteration should be completely independent.

- Parallel kernels are expressed using the parallel_for.

Serial execution

```
for(int i=0; i < 1024; i++){
    a[i] = b[i] + c[i];
});
```

Offload to accelerator using **parallel_for**

```
h.parallel_for(range<1>(1024), [=](id<1> i){
    A[i] =  B[i] + C[i];
});
```

Basic Parallel Kernels

The functions of the basic parallel kernels are provided by the range, id and item classes.

- range describes the size of the task space.
- item represents a single instance of a kernel function and exposes other functions to the query attributes of the execution range.
- Use the information of item to map each thread to the overall task space.

```
h.parallel_for(range<1>(1024), [=](item<1> item){
    auto idx = item.get_id();
    auto R = item.get_range();
    // CODE THAT RUNS ON DEVICE
});
```

Code Sample

```
// Copy from host(CPU) to device(GPU)
my_gpu_queue.memcpy(device_mem, host_mem, N * sizeof(int)).wait();

// do some works on GPU
// submit the content to the queue for execution
my_gpu_queue.submit([&](handler& h) {
```

Run & output:

patric@gpu:~/course$ ./basic_parafor

Selected GPU device: Intel(R) Iris(R) Xe MAX Graphics [0x4905]

Data Result

0, 2, 4, 6, 8, 10, 12, 14, 16, 18,

Task Done!

```
my_gpu_queue.memcpy(host_mem, device_mem, N * sizeof(int)).wait();
```

https://github.com/pengzhao-intel/oneAPI_course/blob/main/code/basic_parafor.cpp

Execution model

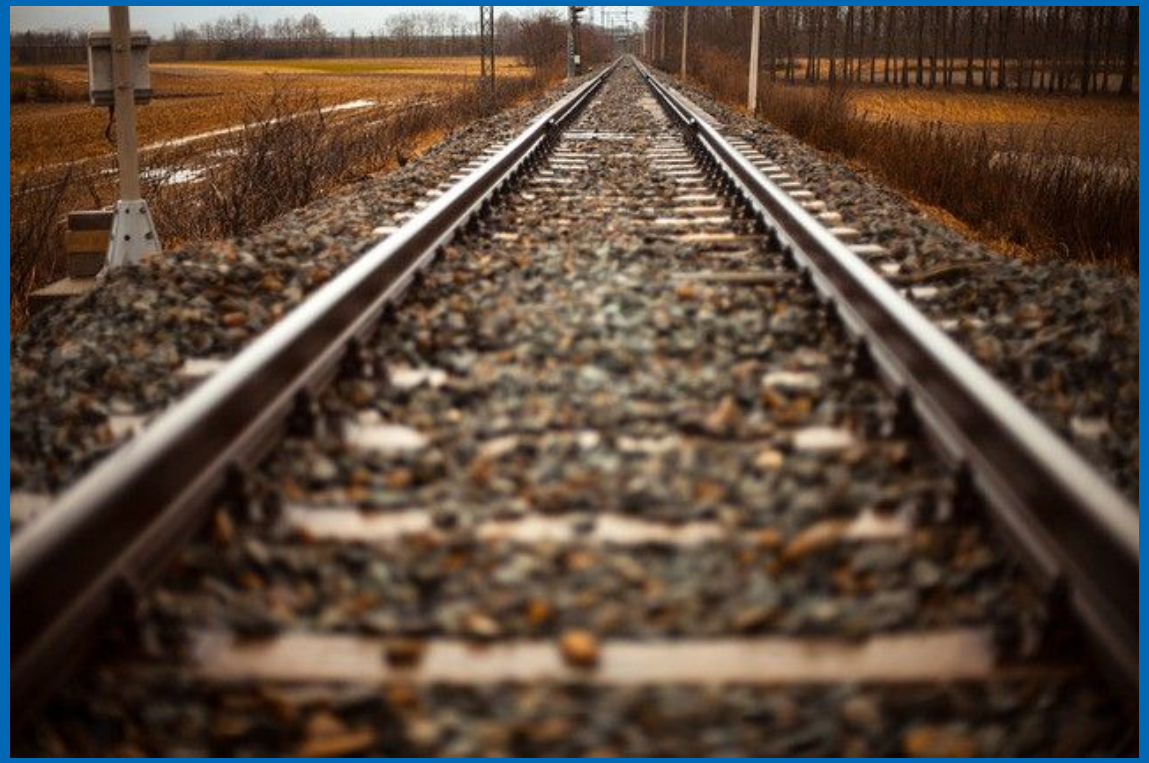


Image Source : http://alana.io/downloads/rails-3/

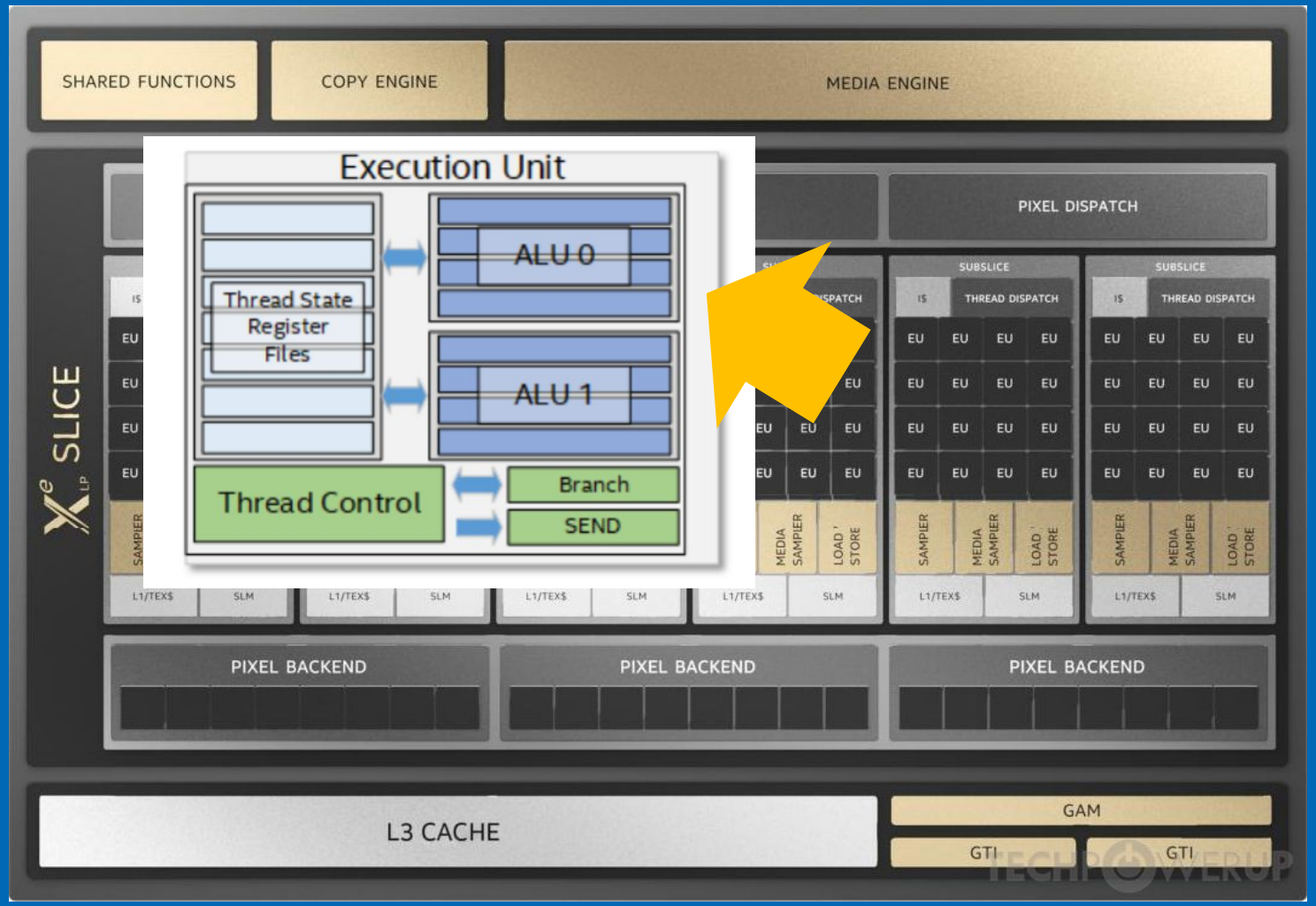
Further Thoughts

- How does the GPU work?
- How does parallel_for work?
- How are tasks mapped to the GPU?
- How do we control the mapping of tasks?

Intel ® Xe Max GPU architecture

Intel SubSlice == NVIDIA SM
Intel Slice == NVIDIA GPC

Render Config	
Shading Units:	768
TMUs:	48
ROPs:	24
Execution Units:	96
FP16 Units:	1536
FP64 Units:	192
Subslices:	6
Slices:	1
L2 Cache:	1024 KB
L3 Cache:	16 MB
Max. TDP:	300 W



DPC++ Task Granularity

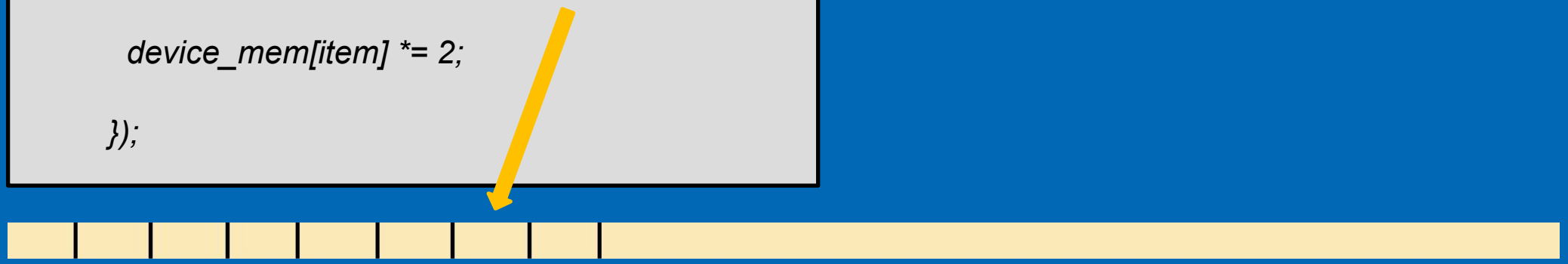
Work-item:

The most basic computing granularity, which will be mapped to the ALU for execution.

Work-group:

Organize a certain number of work-items to be executed at the same time, which will be scheduled to the same subslice, with the work-time sharing certain resources, enabling collaboration within the group.

```
h.parallel_for(range{N}, [=](id<1> item) {
    device_mem[item] *= 2;
});
```



Explicitly set computing granularity

The basic parallel kernel is an easy approach to parallelize for-loops, but lacks precise control.

ND-Range kernel parameter is an enhanced parallel representation method.

- More precise control of task division
- Correspond the execution of the task with the corresponding hardware computing unit

sycl::nd_range

```
template <int dimensions = 1>
class nd_range;
```

The nd_range defines the index space for a work group as well as the global index space. It is passed to parallel_for to execute a kernel on a set of work items.

Template parameters

dimensions	Number of dimensions

https://docs.oneapi.io/versions/latest/dpcpp/iface/nd_range.html#nd-range

ND-Range kernel

The function of the ND_Range kernel is represented by nd_range and nd_item

- The nd_range class represents global execution range and number of work items in each work-group

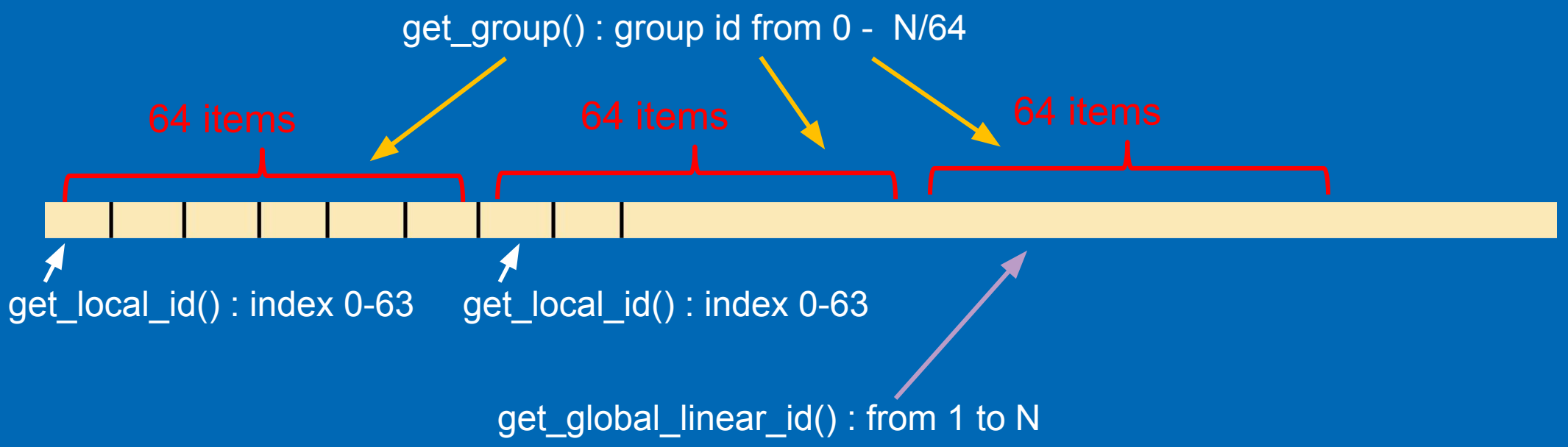
```
h.parallel_for(range{N}, [=](id<1> item) {
    device_mem[item] *= 2;
  });

h.parallel_for(nd_range<1>(N, 64), [=](nd_item<1> item){
  auto idx = item…..;
    device_mem[idx] *= 2;
});
```

How many work-item in each work-group?

How many work-group will be generated?

The nd_item class represents a single instance of a kernel function and allows querying the workgroup range and index.

```
h.parallel_for(nd_range<1>(N,  64), [=](nd_item<1> item){
      auto idx = item. (check if you understand        );
      device_mem[idx] *= 2;
});
```

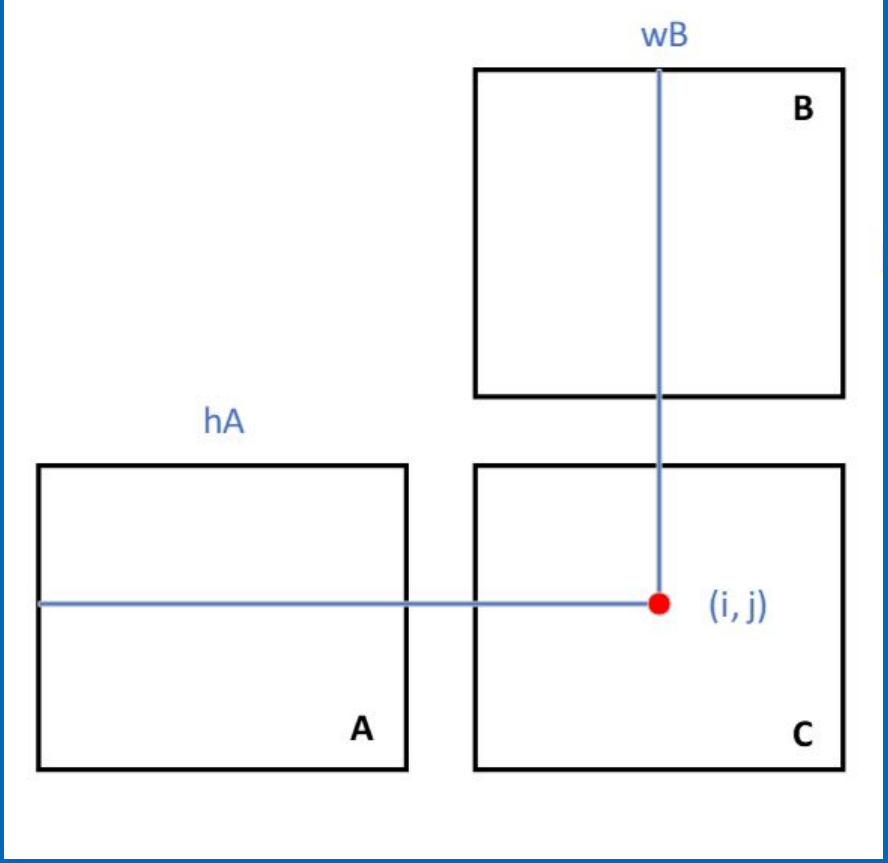


Example: GEMM

Let's show how strong you are!

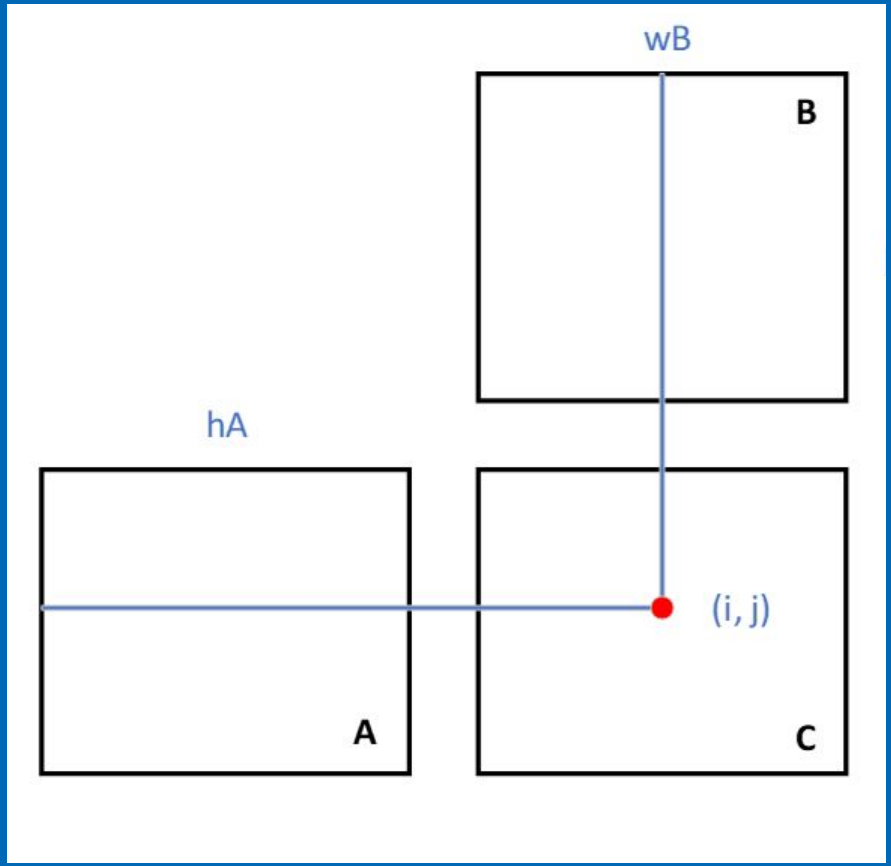
C[i][j] += A[i][hA] * B[hB][j]

```
for (i = 0; i < M; ++i)
    for (j = 0; j < N; ++j)
        for (k = 0; k < L; ++k)
            c[i][j] += a[i][k] * b[k][j];
```

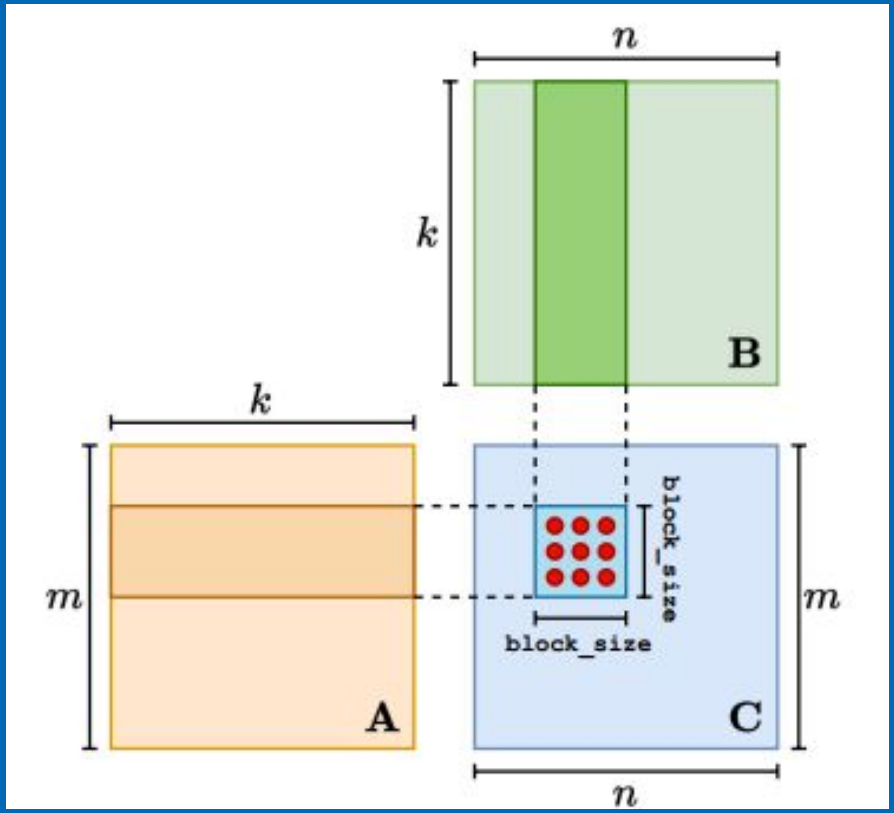


GEMM: CPU Version

```
// Single Thread Computation in CPU
for(int i = 0; i < M; i++) {
    for(int j = 0; j < N; j++) {
        float sum = 0.0f;
        for(int k = 0; k < K; k++) {
            sum +=  cA[i * K + k] * cB[k * N  + j];
        }
        cC[i * N + j] = sum;
    }
}
```



GEMM: GPU version

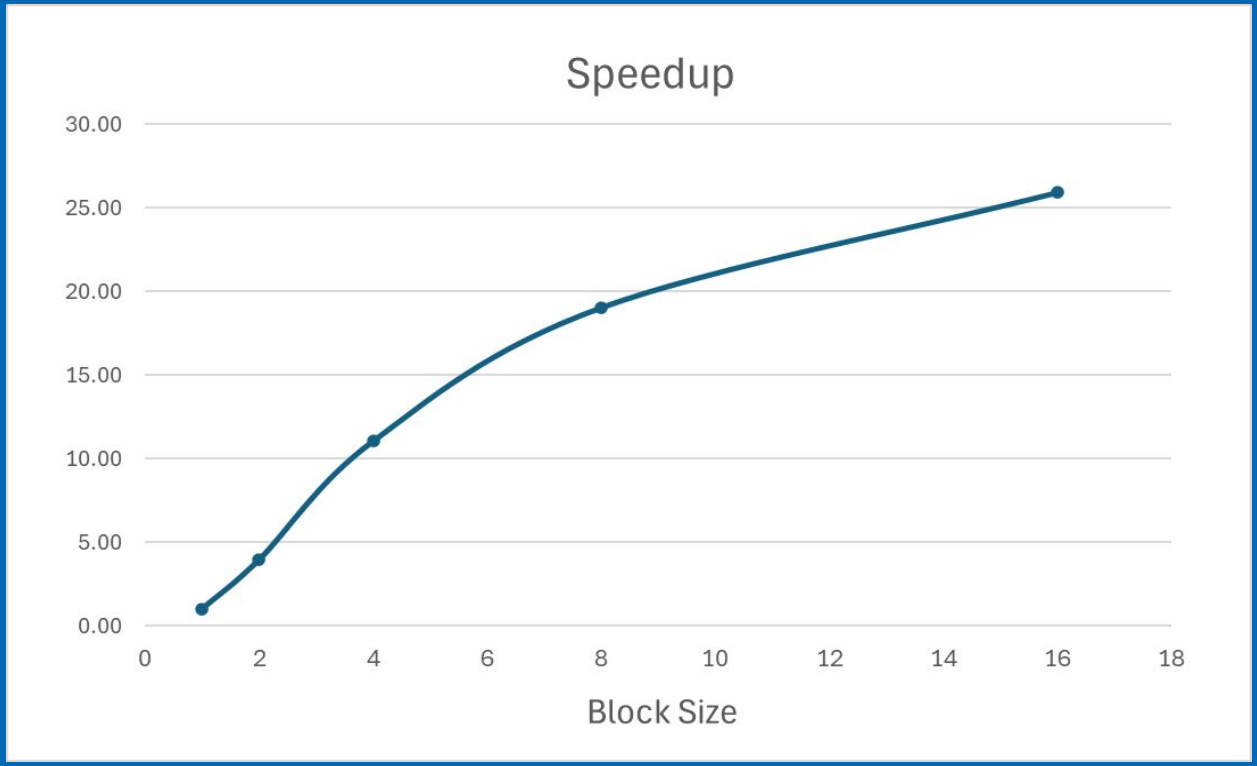


https://github.com/pengzhao-intel/oneAPI_course/blob/main/code/gemm_basic.cpp

```
// define the workgroup size and mapping
  auto grid_rows = (M + block_size - 1) / block_size * block_size;
  auto grid_cols = (N + block_size - 1) / block_size * block_size;
  auto local_ndrange  = range<2>(block_size, block_size);
  auto global_ndrange = range<2>(grid_rows, grid_cols);

  auto e = q.submit([&](sycl::handler &h) {
     h.parallel_for<class k_name_t>(
        sycl::nd_range<2>(global_ndrange, local_ndrange),
                                  [=](sycl::nd_item<2> index) {
           int row = index.get_global_id(0);
           int col = index.get_global_id(1);
           float sum = 0.0f;
           for (int i = 0; i < K; i++) {
             sum += A[row * K + i] * B[i * N  + col];
           }
           C[row * N + col] = sum;
        });
   });
   e.wait();
```

Performance and Occupancy

How does the size of a work-group relate to performance?



Memory model

Image Source : https://www.gocivilairpatrol.com/media/cms/Leadership_Concept__Slides_EBCF53691D635.pdf

Memory hierarchy

- Private Memory

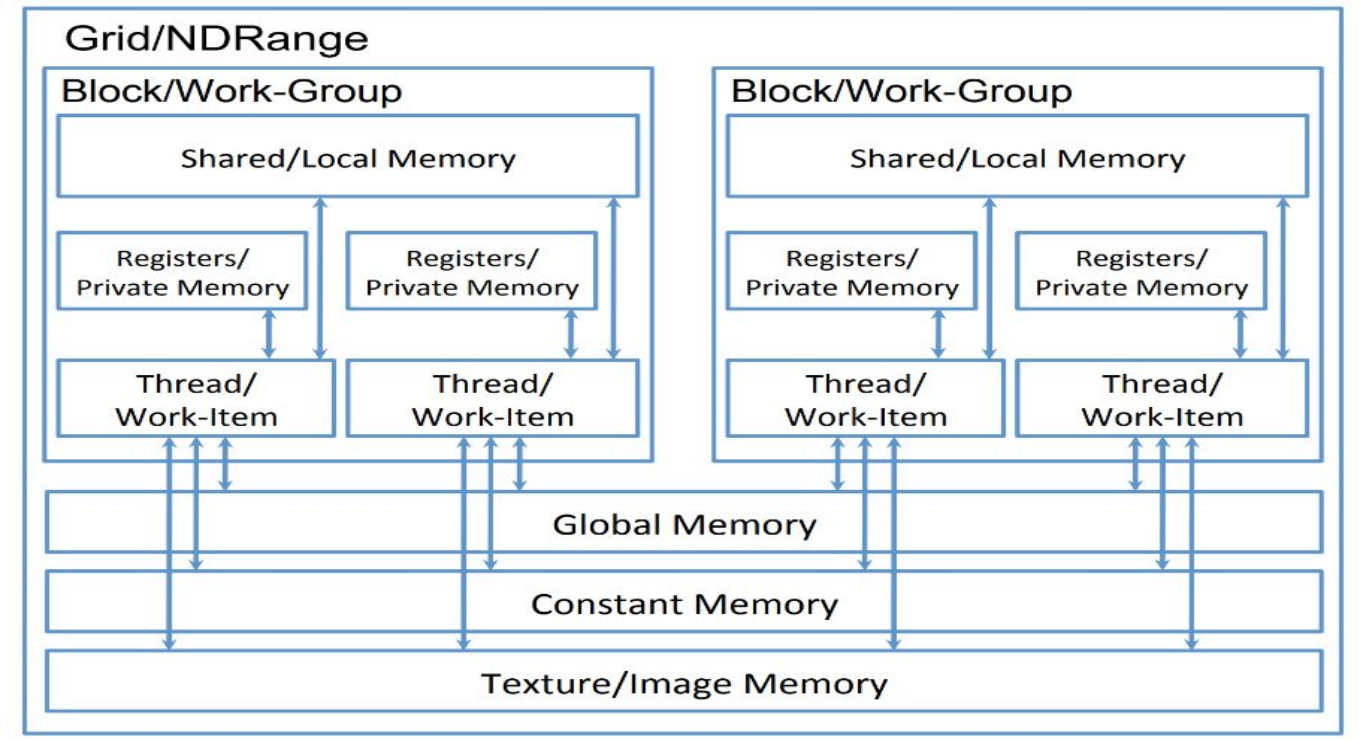
  Thread-independent resources, complementary and visible

- (Shared) Local Memory

  Visible to group members only

- Global Memory

  Visible to all work-items



Private memory

- Created by the work-item itself;
- Visible only to the current work-item and inaccessible between work-items;
- Each has a different value, although the name is the same.

Global memory

- Transfer from external
- Globally visible, with all workitems have access to the same data, e.g. A[0,5];
- Write to the same address at the same time may lead to uncertainty issues (data race), which requires special attention.

```
auto e = q.submit([&](sycl::handler &h) {
    h.parallel_for<class k_name_t>(
        sycl::nd_range<2>(global_ndrange, local_ndrange),
                                    [=](sycl::nd_item<2> index) {
          int row =  ndex.get_global_id(0);
          int col = index.get_global_id(1);
          float sum = 0.0f;
          for (int i = 0; i < K; i++) {
            sum += A[row * K + i] * B[i * N  + col];
          }
          C[row * N + col] = sum;
        });
  });
  e.wait();
```

Private memory/Register

- Fastest but very limited in EU
- Keep the intermediate results in registers
- Using too many registers will limit the number of workgroups that can be run in parallel

Global Memory

- Slow and long latency but very huge size
- Data loading and storing
- Reduce the redundant read/write for better performance

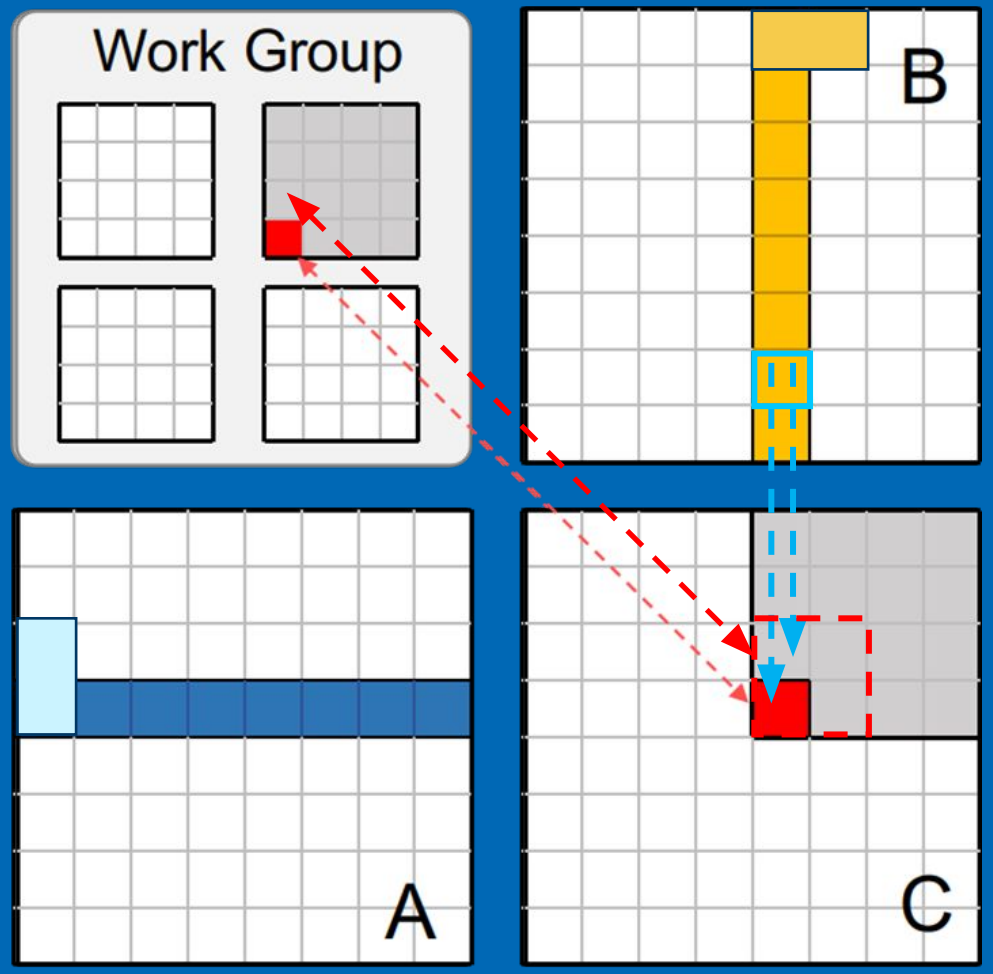
Improve the performance of matrix computation (GEMM)

Data reuse: Global Memory Reuse

- Elements of arrays A and B are reusable in rows and columns
- Data are not reusable when a work-item computes only one output
- It is more efficient for a work-item to compute the output of a tile

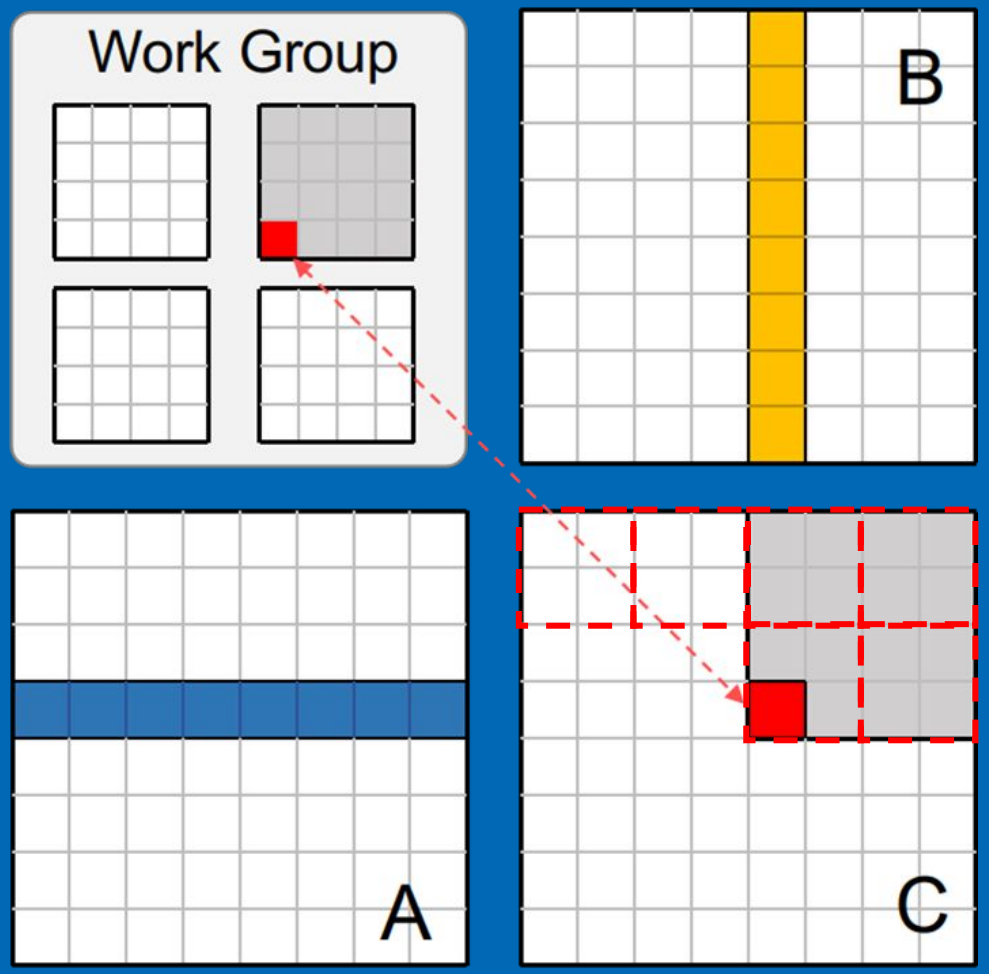
Intermediate results: register

- Read data into register to avoid repeat access to the global memory
- Store the accumulation result in the register of each work-item to avoid repeat read/write to the global memory
- Finally, write out the result of the register in a single shot



GEMM: tile version

```
// Mapping
#define tileX 2
#define tileY 2
// define the workgroup size and mapping
  auto grid_rows = M / tileY;
  auto grid_cols   = N / tileX;
  auto local_ndrange  = range<2>(BLOCK, BLOCK);
  auto global_ndrange = range<2>(grid_rows, grid_cols);

auto e = q.submit([&](sycl::handler &h) {
      h.parallel_for<class k_name_t>(
         sycl::nd_range<2>(global_ndrange, local_ndrange),
                                    [=](sycl::nd_item<2> index) {

            int row = tileY * index.get_global_id(0);
            int col  = tileX * index.get_global_id(1);
             ….
}
```



https://github.com/pengzhao-intel/oneAPI_course/blob/main/code/gemm_tile.cpp

```
float sum[tileY][tileX] = {0.0f};
 float subA[tileY] = {0.0f};
 float subB[tileX] = {0.0f};

  // core computation
 for (int k = 0; k < N; k++) {
            // read data to register
            for(int m = 0; m < tileY; m++) {   subA[m] = A[(row + m) * N + k];      }
            for(int p = 0; p < tileX; p++)    {   subB[p] = B[k * N + p + col];      }

            // math
            for (int m = 0; m < tileY; m++) {
            for (int p = 0; p < tileX; p++) {
                        sum[m][p] += subA[m] * subB[p];
             } }
 } //end of K

 // write results back
 for (int m = 0; m < tileY; m++)
      for (int p = 0; p < tileX; p++)
            C[(row + m) * N + col + p] = sum[m][p];
```

BLOCK	basic	tile 2x2
1	50.85	15.13
2	12.84	4.58
4	4.60	1.97
8	2.67	1.16
16	1.96	1.11

Performance Optimization

Image Source : https://www.constructioninsure.co.uk/high-risk-trade-insurance-most-beneficial/

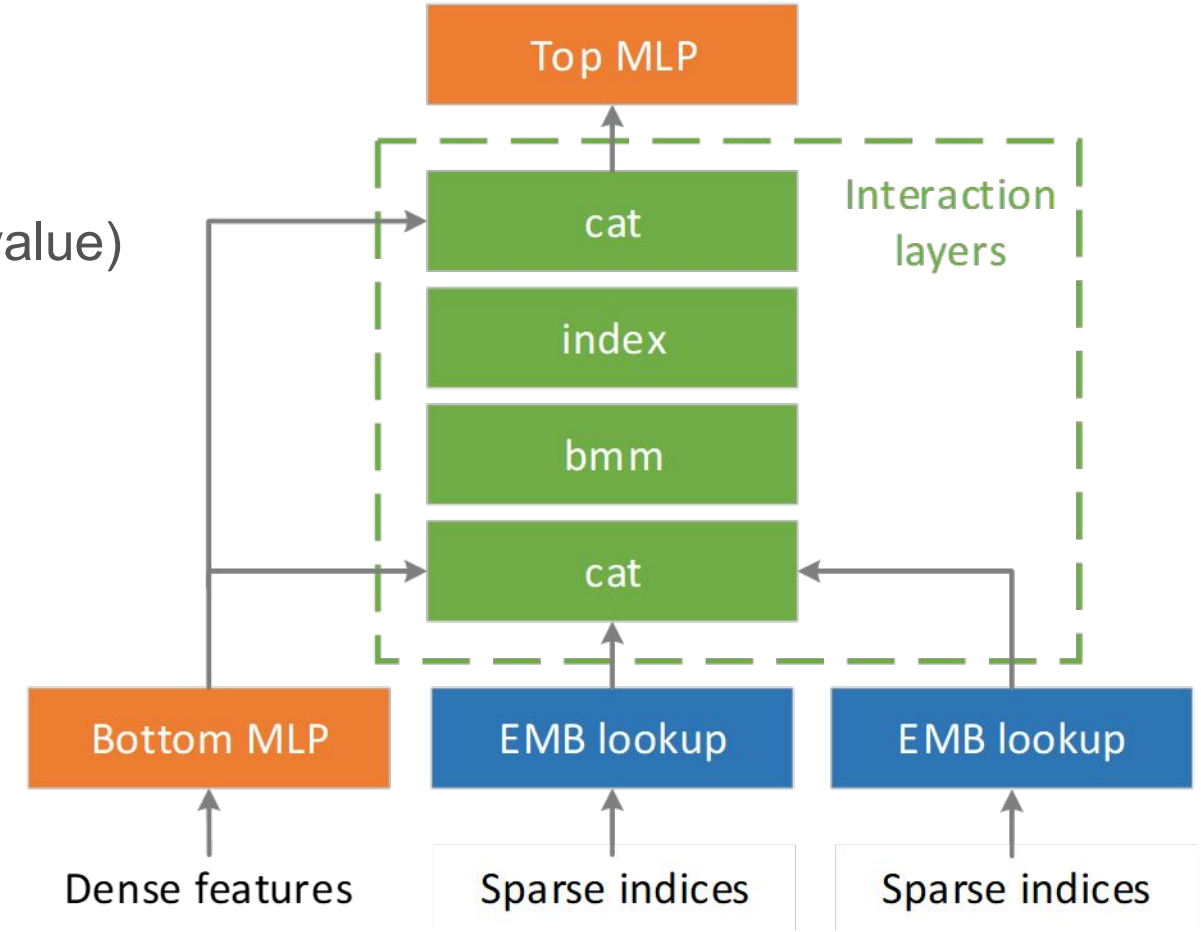
Case Study: Interaction Layers in DLRM

- Inputs:
  - Dense features (128-dim)
  - Sparse features (26 one-hot encoded value)

Output:

- Probability of Click

- DLRM modules:
  - Bottom MLP (matmul)
  - EMB lookup (memory copy)
  - Interaction layers
  - Top MLP (matmul)



Interaction Module Fusion

How does interaction layers work?

- **(Con)Cat**

  cat_res = torch.cat(tensor_list, dim=1)

  Tensor_list: 27 tensors in shape of [32768, 128]

  Cat result shape [32768, 27, 128]

- **Bmm**

  bmm_res = torch.bmm(cat_res, cat_res^T)

  bmm result shape [32768, 27, 27]

- **Index**

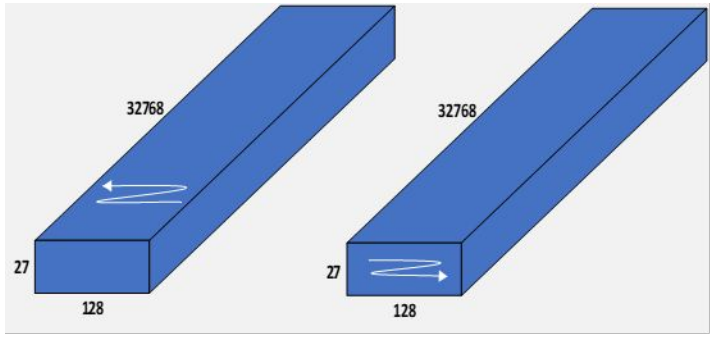
  index_res = bmm_res[: , k , j]

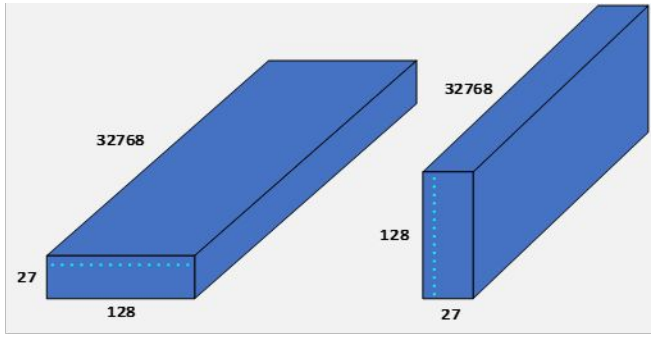
  Index result shape [32768, 351]

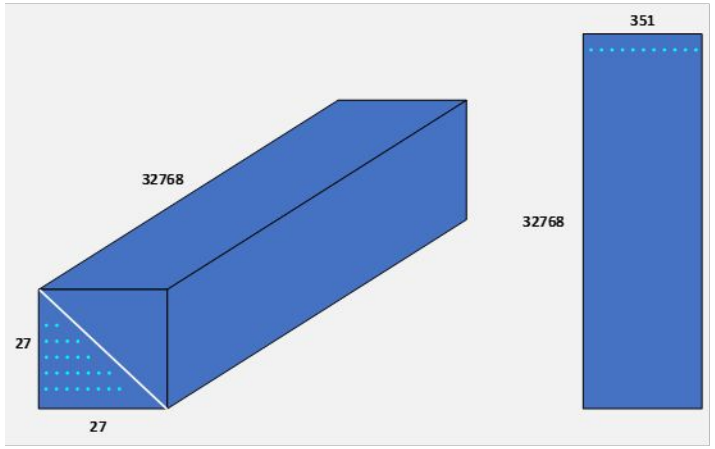
- **(Con)Cat**

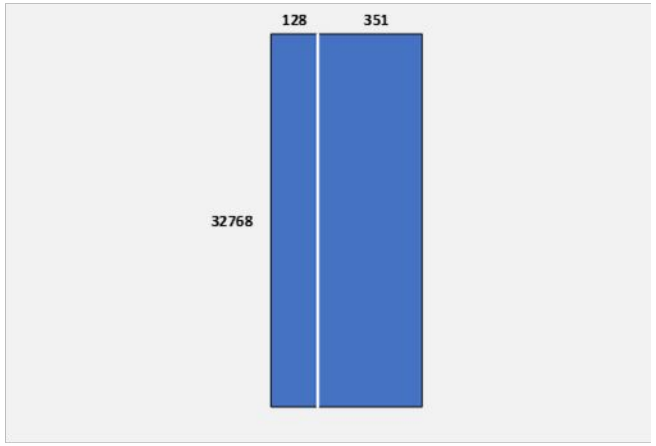
  cat_res = torch.cat(tensor1, index_res)

  cat result shape [32768, 128+351]









▪ Analysis Calculation for BMM

Do we need Systolic Array (DPAS) ?

Do we need FP16/BF16?

ATS	Shape	OP	MEM	OP/Elem	data type	EU Time	DPAS	Mem time (ms) under 700 GB/s	Max
BMM	32768x27x128 * 32768x128x27 -> 32768x27x27	6115295232	250380288	24.42	FP32	0.5688		2.044	2.044
					FP16	0.2844	0.0355	1.022	1.022

Ideally, it's Max(com, mem)

Reality is [Max, Sum(com, mem)]

OP = output * calculate * FMA = 32768*27*27*128*2

MEM = read + write = 32768 * (27*128*2 + 27*27)

EU time (FP32) = OPs / (EU * SIMD width * 2 Op/cycle * Freq * EU eff)

= 6115295232/ (480*8*2*1400 *1e6)*1000

DPAS (DPAS) = OPs / (EU * Systolic Width * Systolic Depth * 2 * 2 Op/cycle * Freq * EU eff)

= 6115295232/ (480*8*8*2*2*1400 *1e6)*1000

Mem time(FP32) = MEM * data type / (Bandwidth * BW Eff) = 250380288 * 4 / 700

Init Design

- Start from BMM

  Because it’s the core part of the whole interaction layers

- BMM: Batch Matrix Multiplication

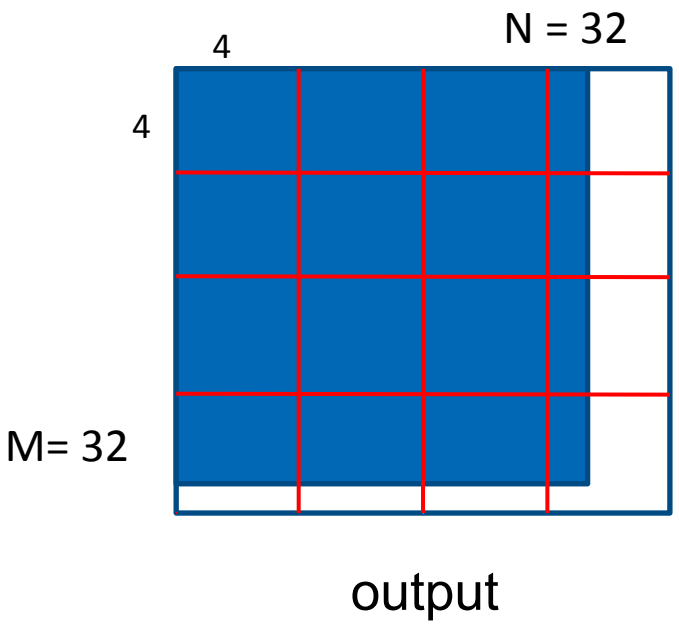
✔ Input 32768x27x27, FP32 datatype

✔ Each workgroup compute 1 batch of output

✔ Padded (virtual) work-item is set to 32x32, but the data is still 27x27

✔ Output Tiling for each work-item is 4x4 submatrix

✔ Global memory loading, computing and storing directly



Begin Optimization

Low Precision

FP32 .vs. FP16

Active from 29.0% to 45.3%

Note:

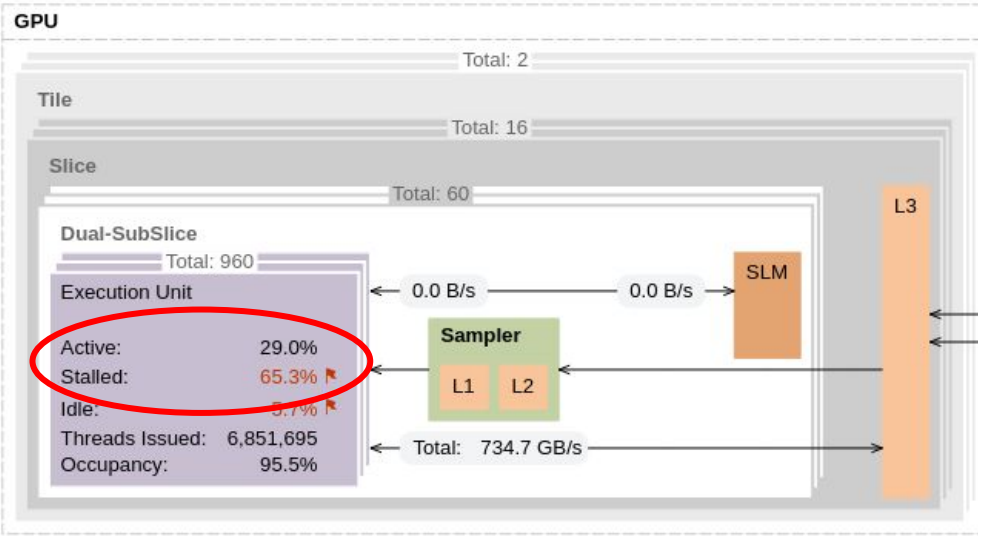
The instruction of FP16 is also Mad – fmad

- ✔ FP32 mad -- 2 cycles
- ✔ FP16 mad -- 1 cycle

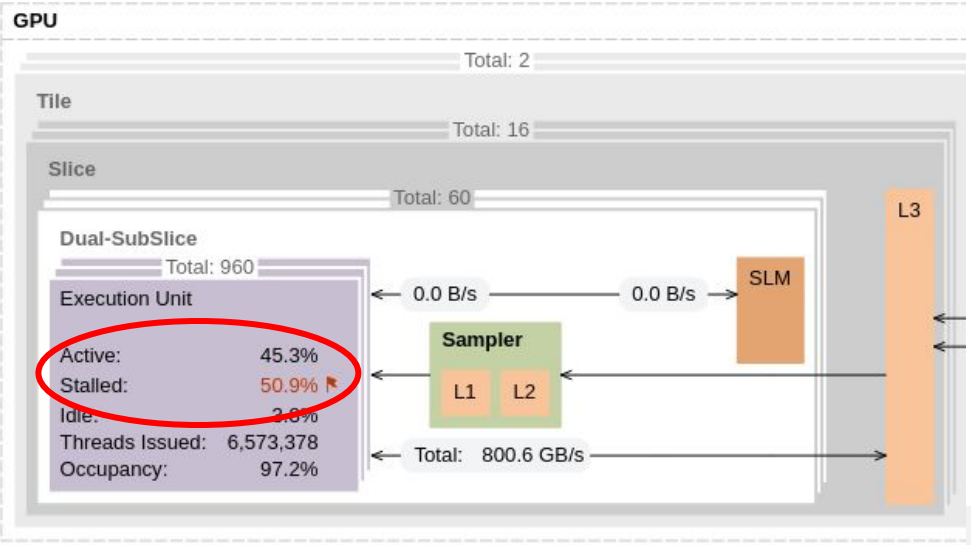
The CUDA instruction is different

- ✔ FP32 fma -- 1 cycle
- ✔ FP16 hfma -- 1 cycle
- ✔ FP16 hfma2 -- 1 cycle but 2 compute inside

FP32



FP16



Mem: FP32 .vs. FP16

Half memory consumption

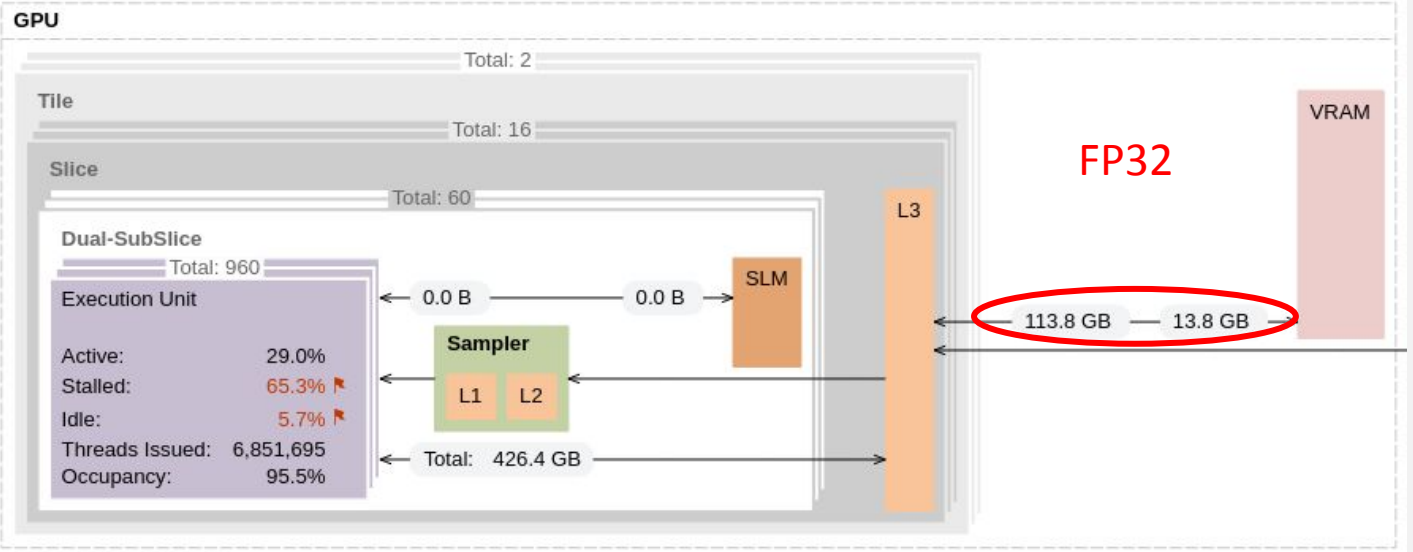
HBM from GMEM to EU

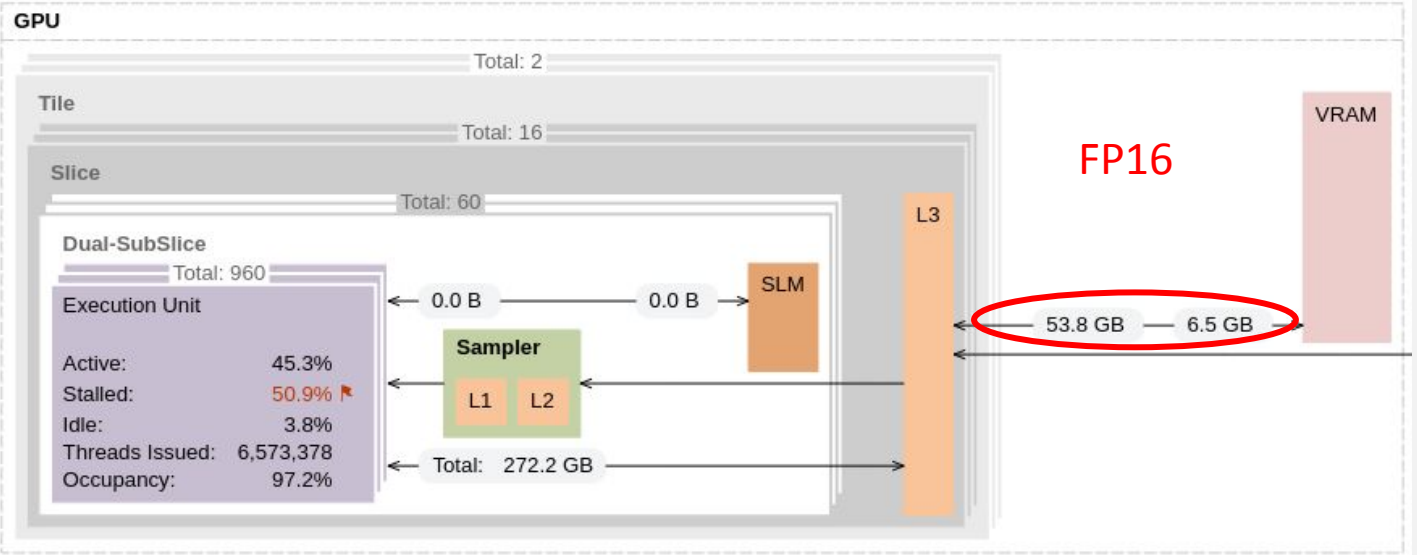
R : 113.8GB -> 53.8GB

W: 13.8GB -> 6.5 GB

L3 read byte

426.4GB -> 272.2GB





Problem size: batch c(32768,32,32) = a(32768,32,128) * b(32768,128,32)

Performance Params: TILE_X = 8 TILE_Y = 8 TILE_CX = 4 TILE_CY = 4

perf	%ops%	%prb_bytes%	%time%	%gflops%	%bandwidth%
perf	8589934592	8192	5.2906	1623.6264	0.0014

Number of 33554432 data are verified.

Test Passed

(base) gta@DUT3014-ATS:~/Patric/gpu-perf-show/codex/build$ ./src/sycl-bmm-bmm-fp16-v1-cpp

Problem size: batch c(32768,32,32) = a(32768,32,128) * b(32768,128,32)

Performance Params: THREAD_X = 8 THREAD_Y = 8 TILE_CX = 4 TILE_CY = 4

perf	%ops%	%prb_bytes%	%time%	%gflops%	%bandwidth%
perf	8589934592	8192	3.0827	2786.5310	0.0025

Number of 33554432 data are verified.

Test Passed

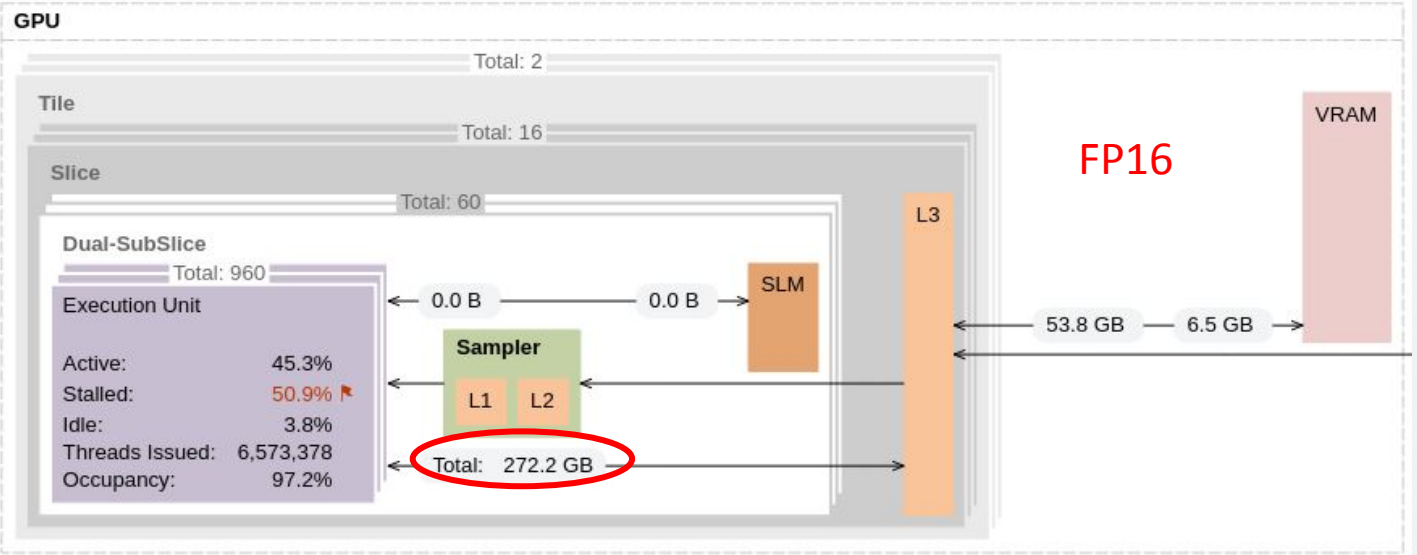
Data Exploration

What can we learn from data?

L3 read byte
426.4GB -> 272.2GB

But L3 read is very larger, 3-5X than HBM
FP32: 3.7X -> 426.4GB/113.8GB
FP16: 5.0X -> 272.2GB/53.8GB

What does this mean?



FP16: SLM version

```
// load data to SLM
int colA = item_linear % TILE_K_32;
int rowA = item_linear / TILE_K_32;
    int strideA = total_thread / TILE_K_32;
    for(int kk = 0; kk < (M * TILE_K_32 / total_thread); kk++) {
     Asub[rowA + kk * strideA][colA] = dA[bs * size_a_32 + (rowA + kk * strideA) * K_32 + k + colA];
}

int colB = item_linear % N_32;
int rowB = item_linear / N_32;
int strideB = total_thread / N_32;
for(int kk = 0; kk < (N_32 * TILE_K / total_thread); kk++) {
     Bsub[rowB + kk * strideB][colB] = dB[bs * size_b_32 + (rowB + (2 *k + kk * strideB)) * N_32 + colB];
}
block.barrier(sycl::access::fence_space::local_space);
```

Why HBM BW is improved?

Mem:

SLM bandwidth: 448GB/s

HBM bandwidth improved:

from 158GB/s -> 202GB/s

Why 'EU Active' is improved?

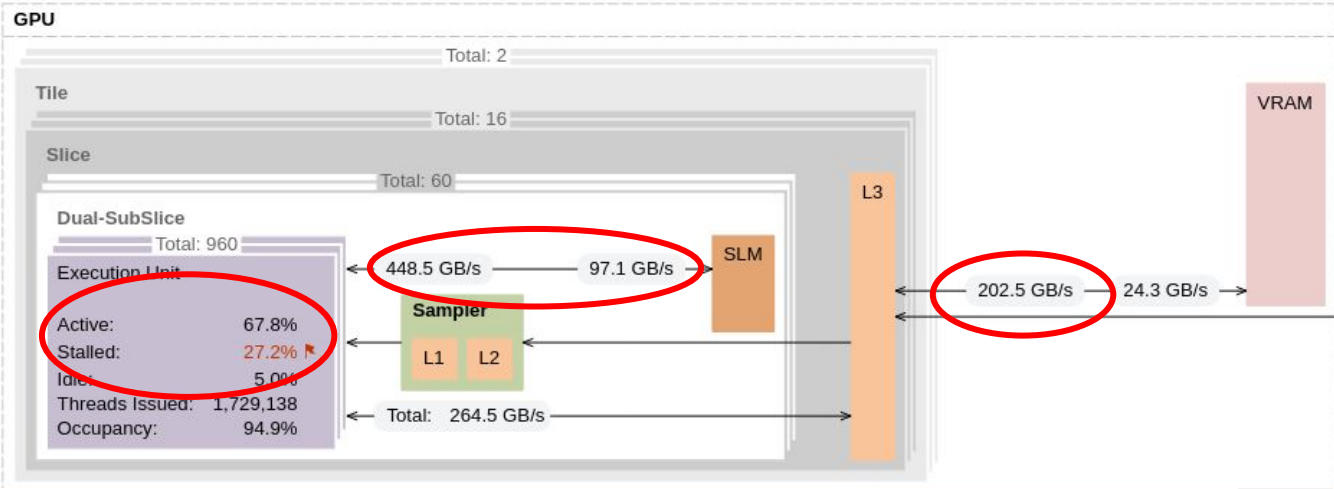
EU:

Active improved from 45.3% to 67.8%

Stalled reduced from 50.9% to 27.2%

FP16 w/o SLM

FP16 w/ SLM



FP16, SLM version, L3

HMB:

Read number doesn't changed

L3:

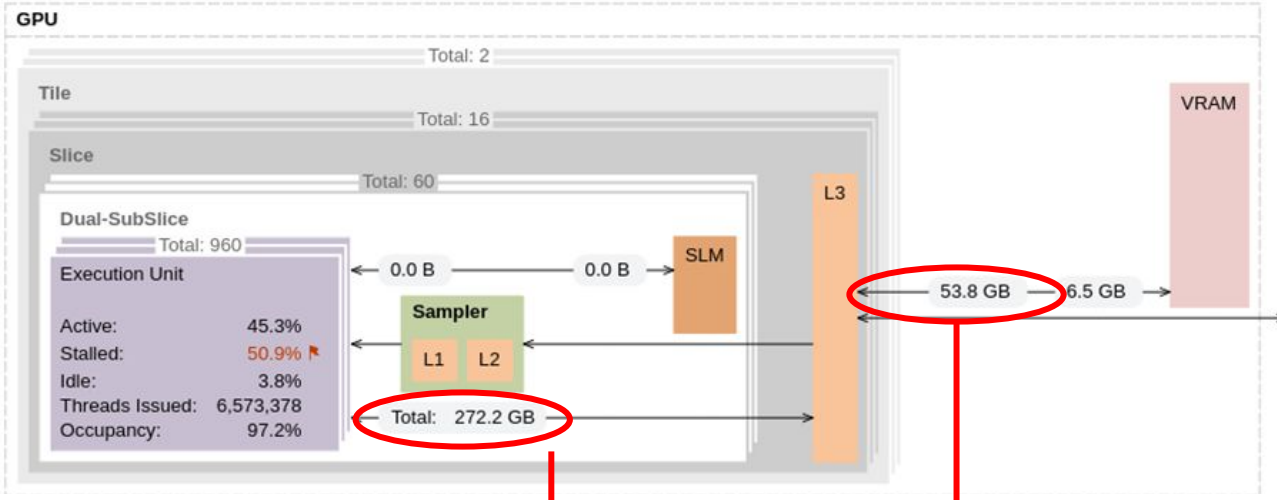
Read number is similar with HBM

We don't go back L3 a lot now!

SMEM:

Repeat read from SMEM, as our expectation

FP16 w/o SLM



FP16 w/ SLM

Problem size: batch c(32768,32,32) = a(32768,32,128) * b(32768,128,32)

Performance Params: THREAD_X = 8 THREAD_Y = 8 TILE_CX = 4 TILE_CY = 4

perf	%ops%	%prb_bytes%	%time%	%gflops%	%bandwidth%
perf	8589934592	8192	3.0827	2786.5310	0.0025

Problem size: batch c(32768,32,32) = a(32768,32,128) * b(32768,128,32)

Performance Params: THREAD_X = 8 THREAD_Y = 8 TILE_CX = 4 TILE_CY = 4 TILE_K = 32

perf	%ops%	%prb_bytes%	%time%	%gflops%	%bandwidth%
perf	8589934592	8192	2.3721	3621.2668	0.0032

Do more DLRM specific changes now ☺️

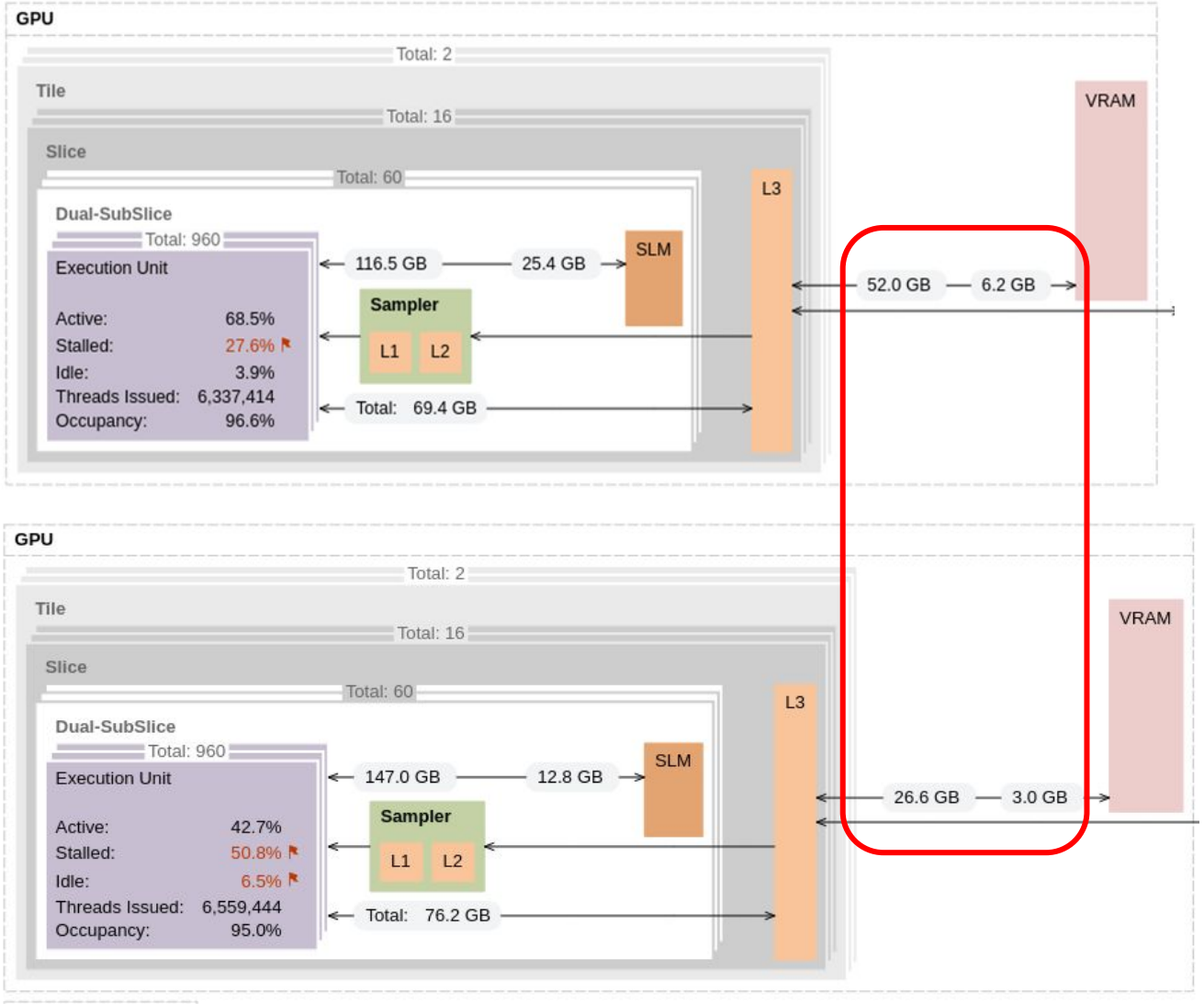
Switch the source A

- One source from A

  memory read number is half

- Fused output of index

  memory output number is half



Where are we going next step?

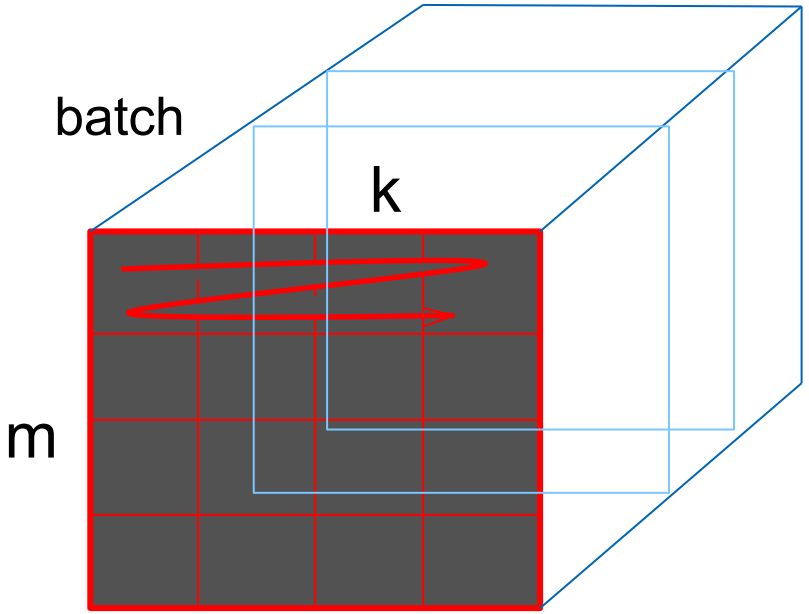
1. Continuous BMM to achieve better performance
2. Fusion others

OP	FWK	Benchdnn	DPCPP	DPCPP (one Source)
cat0	1.49			
bmm	0.94	1.82	2.11	1.42
index	0.52			
cat1	0.30			
total	3.25		3.75	3.21

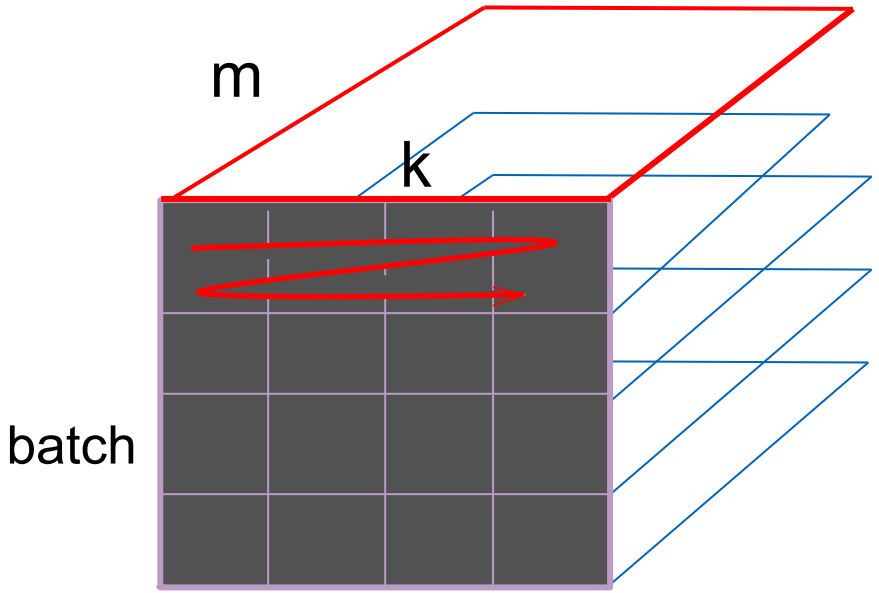
Prefix Fusion for Concate

OP	FWK	DPCPP	DPCPP （bmm + index）	DPCPP （concat + bmm + index）	DPCPP （concat + bmm + index + concat）
cat0	1.49			2.90	3.01
bmm	0.94	1.44	1.43		
index	0.52				
cat1	0.30				
total	3.25	3.75	3.22	3.20	3.01

Data Access Model is changed



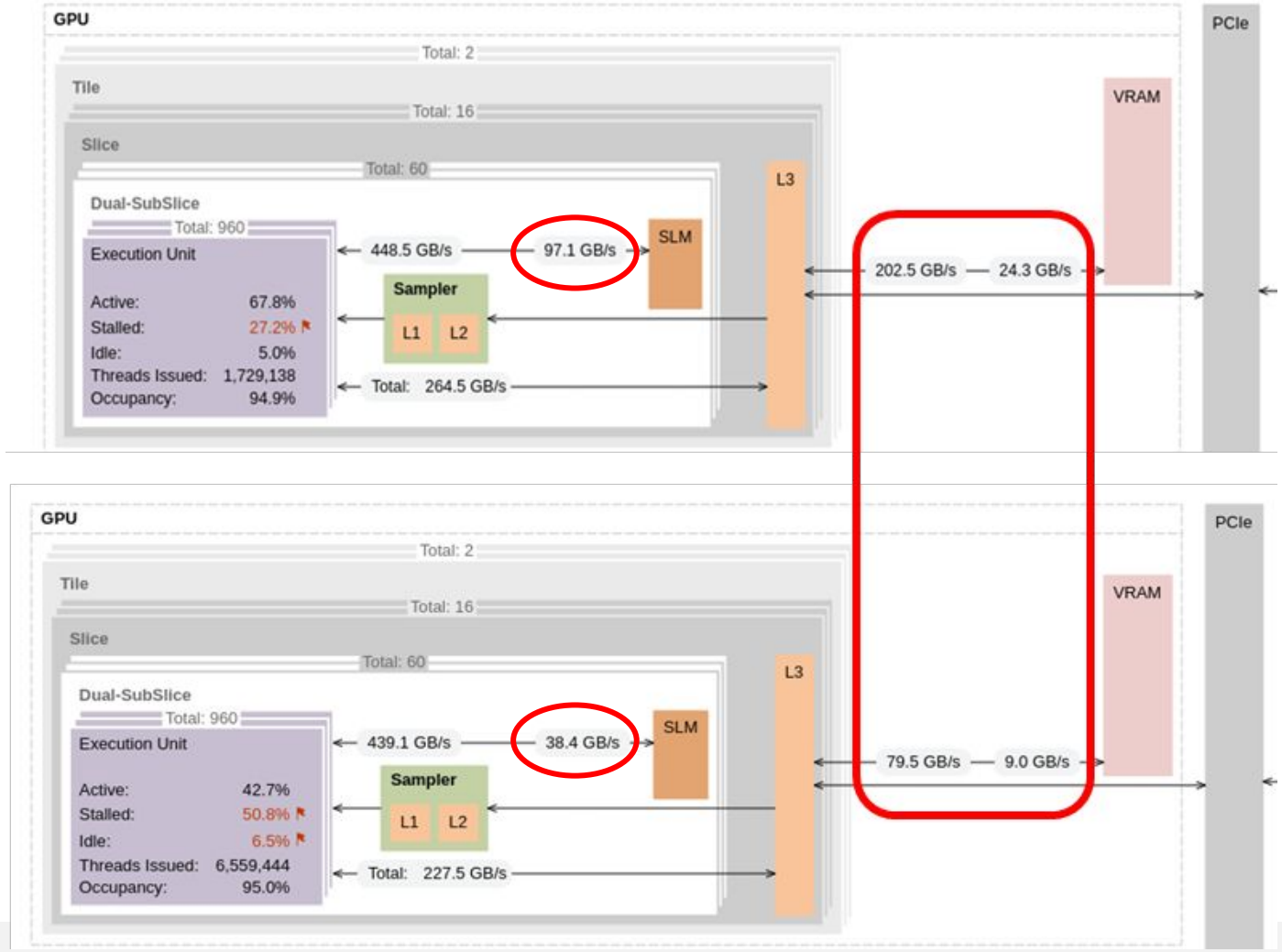
Inputs (batch, m, k) w/ concat (cat)



Inputs (m ,batch, k) w/o concat(cat)

- Bandwidth decreased since the worse locality

How to improve the GMEM bandwidth?

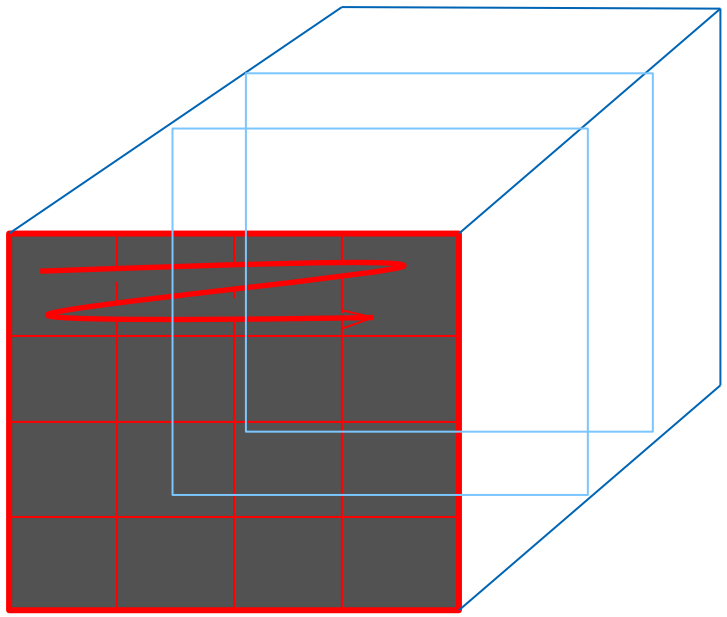


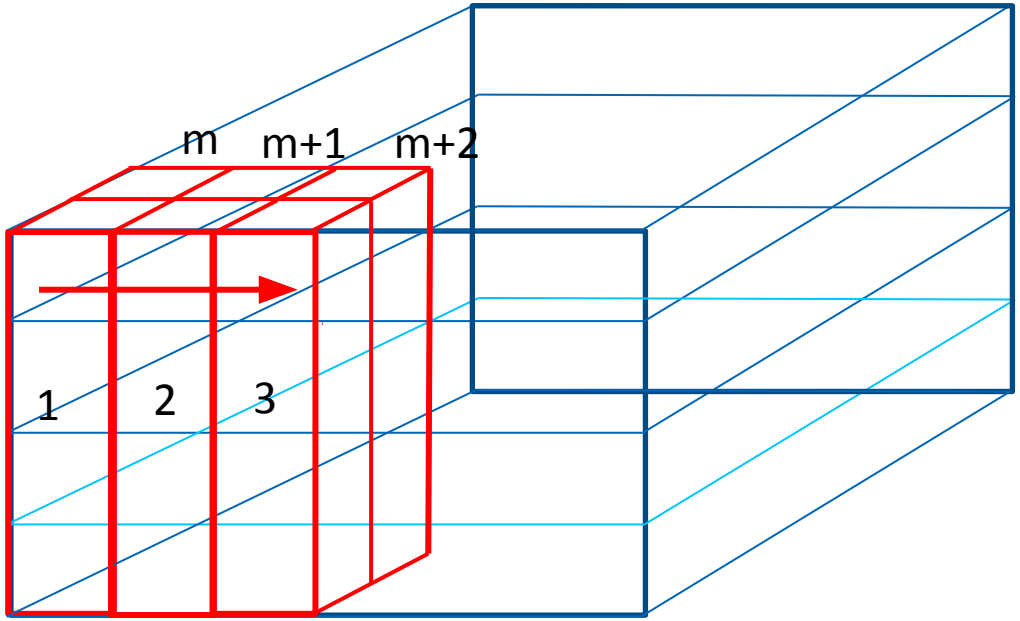
- Tiling in batch size

  reading data in batch direction first, contiguous loading

- Workgroup ordering

  reuse the data from L3 and less data switch



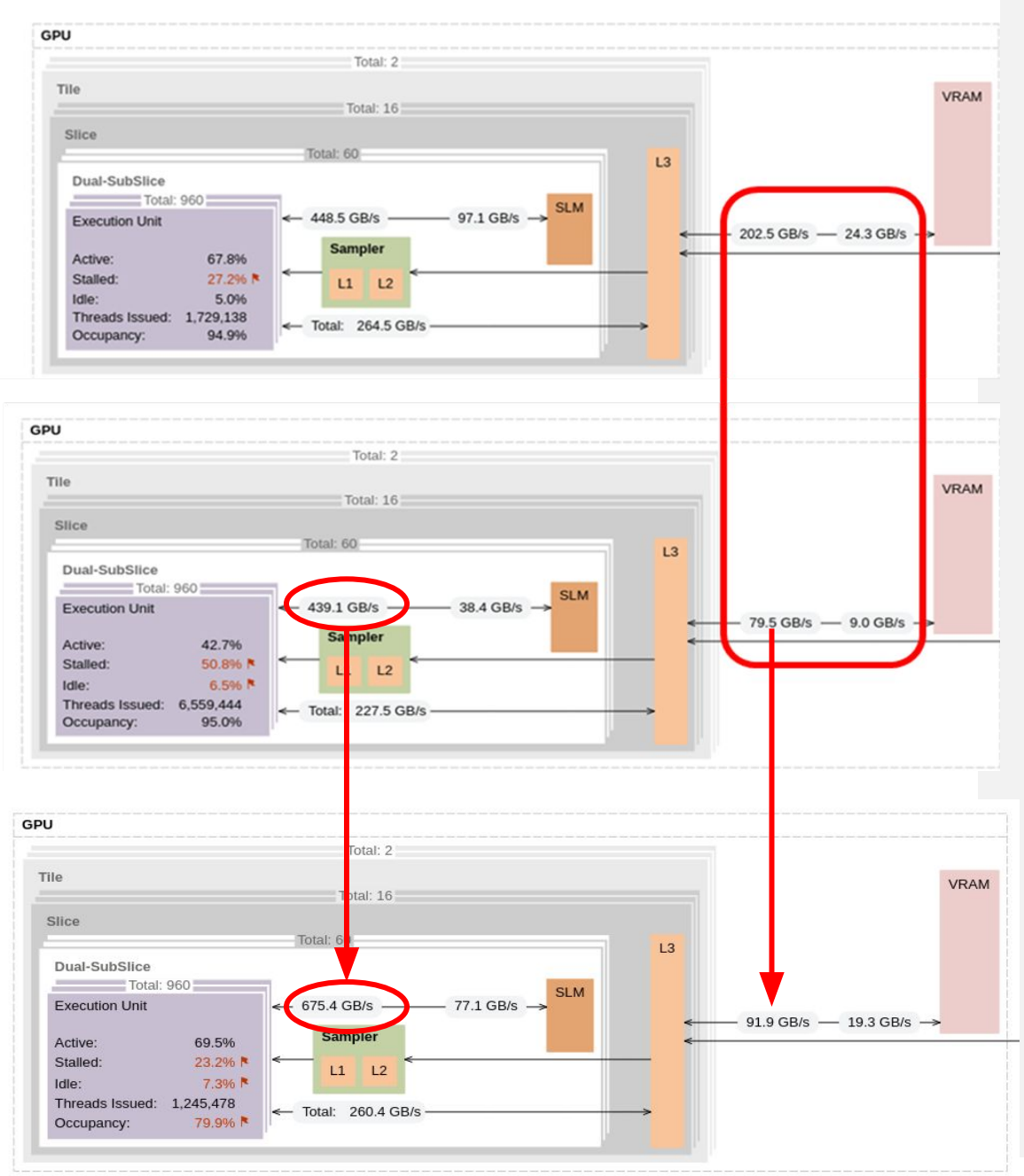


- Batchsize direction data loading

✔ Improved the GMEM BW

✔ Improved SLM read BW

More work-items and SLM are used so the SLM read BW improved



Fused all together

OP	FWK	DPCPP	DPCPP (bmm + index)	DPCPP (bmm + index + concat)	DPCPP (concat + bmm + index)	DPCPP (concat + bmm + index + concat)	speedup
cat0	1.49				1.68	1.73	1.88
bmm	0.94	1.44	1.43	1.5			
index	0.52						
cat1	0.30						
total	3.25	3.75	3.22	2.99	1.98		

Last concate (cat1)

```
for(int kk=0; kk < DivUp(TILE_K_32 * THREAD_B, num_threads); ++kk){
  int n_tile_k = i_tile_k + kk * stride_k;
  // float* dC_32 = reinterpret_cast<float*>(dC + (group_b + n_tile_k) * size_c + (
  if(n_tile_k < THREAD_B)
  {
    half* a_ptr = reinterpret_cast<half*>(&Asub[n_tile_k][0][col]);
    half* c_ptr = dC + (group_b + n_tile_k) * size_c + (k + col) * 2;
    c_ptr[0] = a_ptr[0];
    c_ptr[1] = a_ptr[1];
  }
}
```

Fused Index

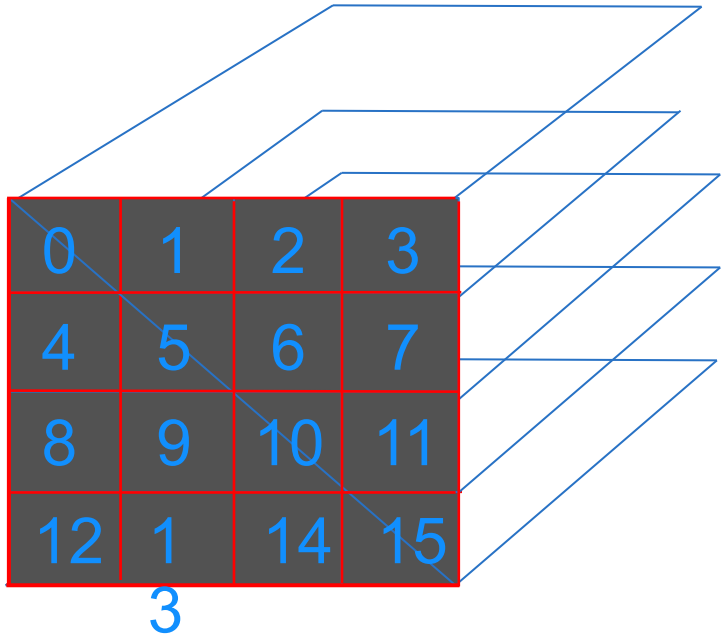
```
        // Write out the result to C
#pragma unroll
        for (int i = 0; i < TILE_CY; ++i) {
          int locate_i = global_i + i;
          if (locate_i >= M) continue;
#pragma unroll
          for (int j = i; j < TILE_CX; ++j) {
            int locate_j = global_j + j;
            if (locate_j >= M) continue;
            if(locate_j < locate_i)
              dC[global_b * size_c + K + locate_i * (locate_i - 1) / 2 + locate_j] = csub[i][j];
          }
        }
```

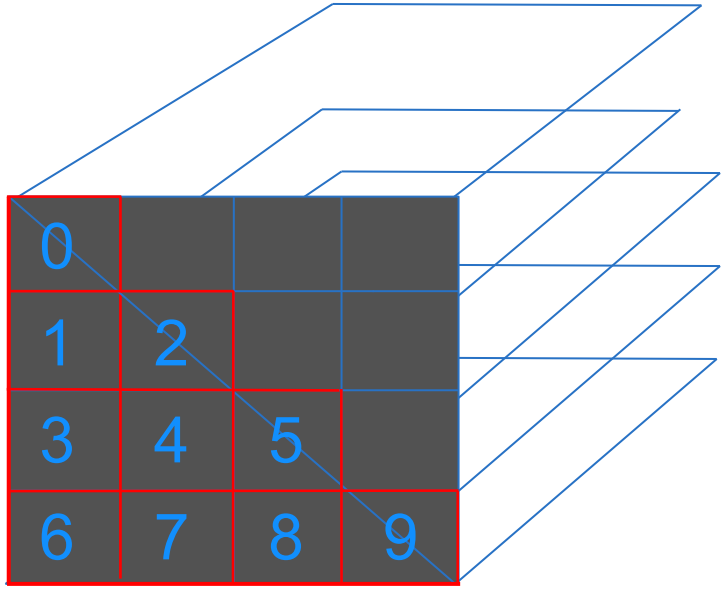
Half Computation

Only half WG is needed, and we changed the workgroup mapping

Work Size	
Global ▼	Local
8 x 8 x 32768	8 x 8 x 1

Work Size	
Global ▼	Local
32 x 32768	32 x 2





Final Performance

<table>
<tr><th>OP</th><th>FWK</th><th>DPCPP</th><th>DPCPP
（bmm + index）</th><th>DPCPP
(bmm + index + concat)</th><th>DPCPP
（concat + bmm + index）</th><th>DPCPP
（concat + bmm + index + concat）</th><th>DPCPP : half computation
（concat + bmm + index + concat）</th><th>speedup</th></tr>
<tr><td>cat0</td><td>1.49</td><td></td><td></td><td></td><td rowspan="3">1.68</td><td rowspan="5">1.87</td><td rowspan="5">1.39</td><td rowspan="5">2.34X</td></tr>
<tr><td>bmm</td><td>0.94</td><td>1.44</td><td rowspan="2">1.43</td><td rowspan="3">1.5</td></tr>
<tr><td>index</td><td>0.52</td><td></td></tr>
<tr><td>cat1</td><td>0.30</td><td></td><td></td><td></td></tr>
<tr><td>total</td><td>3.25</td><td>3.75</td><td>3.22</td><td>2.99</td><td>1.98</td></tr>
</table>

Thank you and Questions